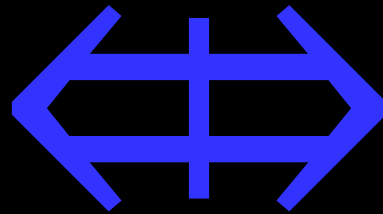


# Dezvoltarea aplicațiilor Web



**servicii Web în stil REST**

Prof. Sabin Corneliu Buraga  
[profs.info.uaic.ro/sabin.buraga/](https://profs.info.uaic.ro/sabin.buraga/)

„Nu spune niciodată «*nu se poate*»,  
ci începe cu «*să vedem*».”

**Nicolae Iorga**



Care sunt scopurile Web-ului?

Constituirea și interacțiunea  
cu un spațiu de comunicare inter-umană  
partajarea cunoștințelor

**Web social** (“**Web 2.0**”) – *users as prosumers*  
(producători și consumatori de informații)

Constituirea și interacțiunea  
cu un spațiu de comunicare inter-umană  
partajarea cunoștințelor

Web al datelor / Web semantic

modelarea și interconectarea (*linked*) datelor/  
informațiilor/cunoștințelor pentru a fi „înțelese”  
și procesate „inteligent” de către entități computaționale

# Exploatarea puterii computaționale

accesul la Web se poate realiza  
via dispozitive având resurse reduse

Web ubicuu (omniprezent):  
Web mobil, Web 3D, WoT – *Web of Things*,...



Cum pot fi accesate și procesate resursele  
– date, informații, cunoștințe –  
disponibile pe Web?

# nevoi ale dezvoltatorilor Web

Soluții multi-platformă, slab-conectate

integrare (în timp-real) la nivel de Internet/Web  
a aplicațiilor, serviciilor și sistemelor



# nevoi ale dezvoltatorilor Web

## Soluții multi-platformă, slab-conectate

exemplificare:

găsirea ofertelor de servicii,  
pe baza localizării geografice a utilizatorului,  
în contextul dispozitivelor mobile

scenariu:

„vreau să comand     disponibile în proximitate,  
produse ce-mi vor fi livrate cât mai rapid la domiciliu”

# nevoi ale dezvoltatorilor Web

Soluții multi-platformă, slab-conectate

datele să poată fi descrise pentru a fi „înțelese”  
de calculatoare și pentru a fi interconectate facil

# nevoi ale dezvoltatorilor Web

## Soluții multi-platformă, slab-conectate

datele să poată fi descrise pentru a fi „înțelese” de calculatoare și pentru a fi interconectate facil

 *Web “puzzles”*

relaționare „inteligentă” a mai multor servicii oferind „lucruri” de interes, conform preferințelor utilizatorului

# nevoi ale dezvoltatorilor Web

## Soluții multi-platformă, slab-conectate

datele să poată fi descrise pentru a fi „înțelese” de calculatoare și pentru a fi interconectate facil



relaționare „inteligentă” a mai multor servicii oferind „lucruri” de interes, conform preferințelor utilizatorului

exemplu: prezicerea apariției unei noi versiuni de joc Web și a prețului său în funcție de contextul curent

# nevoi ale dezvoltatorilor Web

Servicii ataşabile (*pluggable*) & versatile

software (aplicații Web) considerate drept servicii  
*Software as a Service* – SaaS

ofertant de servicii utilizate la nivel de aplicație  
*Application Service Provider* – ASP

# soluție

Divizarea aplicațiilor în **servicii** – independente –  
care se pot **compune**,  
menite a se **conecta** și **orchestra** în mod **spontan**  
în cadrul proceselor de afaceri/tehnice

*Web component-based software*

# soluție

*“The Web is the computer”*

disponibilitatea unei/unor arhitecturi care...

oferă suport pentru paradigme de comunicare  
– bazată pe actualele tehnologii Web –  
între aplicații eterogene

ce recurg la diferite paradigme & limbaje de programare,  
și/sau platforme hardware/software

# soluție

*“The Web is the computer”*

disponibilitatea unei/unor arhitecturi care...

permit(e) localizarea transparentă a serviciilor



# soluție

*“The Web is the computer”*

disponibilitatea unei/unor arhitecturi care...

ascund(e) dezvoltatorului detaliile de sistem

# soluție

*“The Web is the computer”*

disponibilitatea unei/unor arhitecturi care...

asigură calitatea dezvoltării și exploatării  
aplicațiilor distribuite și/sau paralele:  
**standardizare, securitate, disponibilitate,  
reutilizare, mentenanță etc.**



Ce sunt serviciile Web?

# servicii web

*“A **service** is an abstract resource that represents a capability of performing tasks that form a coherent functionality from the point of view of providers entities and requesters entities.”*

[www.w3.org/TR/ws-gloss/](http://www.w3.org/TR/ws-gloss/)

# servicii web

“A **Web service** *is a software system designed to support interoperable machine-to-machine interaction over a network.*”

[www.w3.org/TR/ws-gloss/](http://www.w3.org/TR/ws-gloss/)

# servicii web

Software oferind o funcționalitate specifică

acces la resurse – Instagram, GoodReads, Pinterest, Vimeo

agregare de știri/resurse – Feedly, Pocket, Reddit

cartografiere – Google Maps, OpenStreetMap etc.

mesagerie instantanee – Discord, Telegram, Twilio

procesări – Amazon Rekognition, IBM Watson,

Microsoft Azure Cognitive Services, Google TensorFlow,...

realizare de statistici Web – e.g., Open Web Analytics

*spelling checking* – Bing Spell Check, xSpell,...

stocare de date – Amazon S3, Dropbox, OneDrive etc.

rețele sociale – vezi Facebook Open Graph

...

Search headlines...



All sites ▾ Any time ▾

? Search tips

**M Medium Programming**  
latest • day • week • month

- 18h [The Upgrade Hell](#)
- 2d [The Unauthorized Experiment: How AI Secretly Infiltrated Reddit and Changed Users' Minds](#)
- 2d [Colibri: The Fully Declarative And Turing-Complete Language Lurking Inside Swift's Type System](#)
- 2d [CSS Isn't Broken. You Just Never Read the Docs.](#)
- 3d [De Profundis: Month 1 of 12 in My Year of Deliberate Pain with Rust](#)
- 3d [Stop Hiding Bad Code Behind Good Tooling](#)
- 4d [From Color Blindness Grit to a Simple Tool for Change](#)

**L Lobsters**  
latest • day • week • month

- 2h [Odin, A Pragmatic C Alternative with a Go Flavour](#)
- 2h [Augmentation / Replacement](#)
- 4h [Stop Fighting Your ROS 2 Environment: Build Faster, Reproducibly, Anywhere](#)
- 4h [What are you doing this week?](#)
- 7h [Fuzzing with Grammars](#)
- 8h [modern-latex: A short guide to LaTeX that avoids legacy cruft](#)
- 12h [iBeacons](#)
- 15h [Tech Companies Apparently Do Not Understand Why We Dislike AI](#)

**Hacker Noon**  
latest • day • week • month

- 8h [The TechBeat: Celebrating Vitruvian Man Day at SDG: A Tribute to Arts, Science, & Human Potential \(5/5/2025\)](#)
- 10h [How to Set Up a Free Oracle Cloud VM for Web Development \(2025 Guide\)](#)
- 10h [Spam vs Phishing: Know the Difference](#)
- 10h [Ukraine is a Building a Drone Wall Against Russia](#)
- 10h [Six Side Characters That Stole The Show In TV Shows](#)
- 11h [A Beginner's Guide to Reading and Writing Smart Contracts on Explorers](#)

utilizate – la distanță – de alte aplicații/servicii

|                               |                                  |                                  |                           |
|-------------------------------|----------------------------------|----------------------------------|---------------------------|
| Web Application Development x |                                  |                                  |                           |
| fx                            | =GOOGLETRANSLATE(B1, "ro", "en") |                                  |                           |
|                               | A                                | B                                |                           |
| 1                             | Text                             | B2                               | voltarea aplicațiilor Web |
| 2                             | Translation                      | =GOOGLETRANSLATE(B1, "ro", "en") |                           |
| 3                             |                                  |                                  |                           |

# servicii web

Accesate standardizat via Web

adresare de resurse cu URI

transfer de date via HTTP

mesaje adoptând formate de date: CSV, JSON, XML,...





Cum am implementa un serviciu?

# servicii web

## Implementare standard

recurgerea la servere + *framework*-uri de aplicații Web



ASP.NET MVC, Django, Node.js,  
PHP (CodeIgniter, Laravel, Symfony,...),  
Play, Ruby on Rails, Spring etc.

# servicii web

Tradițional, aplicația oferă o interfață-utilizator disponibilă pe Web

limbaj de marcare – *e.g.*, HTML

stiluri de prezentare a conținutului – CSS

interactivitate via JavaScript (+biblioteci/*framework*-uri)

Search term(s)

web

Add and

Subject

All classifications will be included by default.

☒ Computer Science (cs)

☐ Economics (econ)

☐ Electrical Engineering and Systems Science (eess)

☐ Mathematics (math)

☐ Physics

☐ Quantitative

☐ Quantitative

☐ Statistics (sta)

☒ Include cross-listed papers ☐ Exclude cross-listed papers

Date

☒ All dates

☐ Past 12 months

☐ Specific year

✓ Title

Author(s)

Abstract

Comments

Journal reference

ACM classification

MSC classification

Report number

arXiv identifier

Cross-list category

DOI

ORCID

arXiv author ID

All fields

cererile sunt capt(ur)ate via formulare,  
componente interactive + legături hipermedia

|        |              |
|--------|--------------|
| 33     | Euro         |
| 159.46 | Romanian Leu |
|        | ???          |
| 33     | 유로           |
| 159.48 | 루마니아 레우      |

utilizatorii umani trebuie să interpreteze etichetele și câmpurile de dialog expuse de interfața aplicației Web

# servicii web

Serviciul implementat de aplicație oferă un răspuns  
(o reprezentare a unei resurse Web)

uzual, un document HTML al cărui conținut  
e transferat la client conform unui protocol – HTTP(S)

# servicii web

Serviciul implementat de aplicație oferă un răspuns  
(o reprezentare a unei resurse Web)

uzual, un document HTML al cărui conținut  
e transferat la client conform unui protocol – HTTP(S)

există rare cazuri în care documentele HTML  
sunt scrise/generate corect – *malformed markup*  
aspect de interes: ignorarea erorilor de sintaxă

# servicii web

Cum obținem răspunsul pentru a fi (re)folosit  
în programele noastre?

necesitate: procesarea datelor din codul HTML

## *Web scraping*

uzual, se recurge la biblioteci/*framework*-uri specializate:

Beautiful Soup (Python), HAP (C#), jsoup (Java),  
Masterminds HTML5-PHP, Parse5 (Node.js),  
ScrAPIr (JavaScript), Scrapy (Python), SwiftSoup



# servicii web

Cum obținem răspunsul pentru a fi (re)folosit  
în programele noastre?

```
class="vk_sh c8Zgcf B682Df"><div><span class="DF1fde eNFL1">33</span> <span class="vLqKYe" data-mid="/m/02l6h" data-name="Euro">Euro</span> equals</div><div class="w5HbFd"></div></div><div class="dDoNo ikb4Bb gsrt GDBPqd"><span class="DF1fde SwHCTb" data-precision="2" data-value="164.28225">164,28</span> <span class="MWvIVE" data-mid="/m/02zsyq" data-name="Romanian Leu">Romanian Leu</span></div></div><div class="k0Rg6d
```

???

orice modificare în marcaje duce la rescrierea programului  
de preluare a datelor din documentul HTML

# servicii web: caracterizare

Serviciile Web fac explicite specificațiile implicite

datele de intrare și răspunsul pot fi specificate (riguros)  
via diverse maniere de validare

exemple: JSON Schema, XML Schema,  
pentru date RDF: SHACL (*Shapes Constraint Language*)

# servicii web: caracterizare

Utilizate la interacțiunea dintre aplicații

dinamice

lipsa unei cunoașteri *a-priori* a interacțiunii  
cu alte aplicații/servicii Web

# **servicii web: caracterizare**

Puncte finale utilizate pentru procesarea datelor,  
în manieră publică – eventual, via API-uri deschise

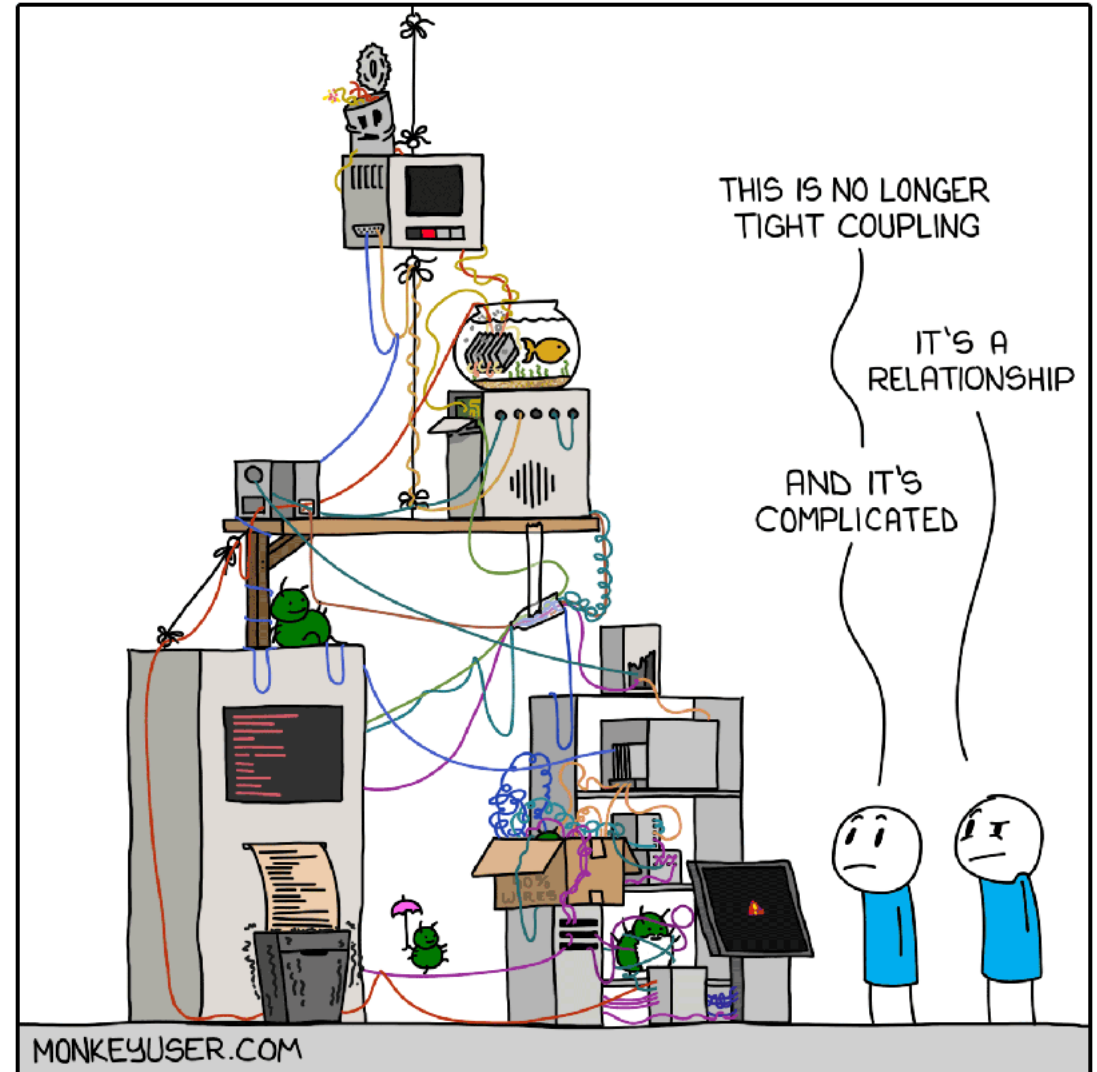
# **servicii web: caracterizare**

Dezvoltate pe baza platformelor, arhitecturilor,  
tehnologiilor și limbajelor curente



Există un model  
arhitectural de  
dezvoltare  
a serviciilor  
la nivel de Web?

NEXT LEVEL



**soa**

Arhitectura orientată spre servicii

*Service Oriented Architecture*

# soa

**Stil arhitectural** de proiectare și dezvoltare  
de aplicații considerate drept servicii  
care pot fi invocate de alte aplicații

T. Erl, *Service-Oriented Architecture:  
Analysis & Design for Services and Microservices* (2<sup>nd</sup> Ed.),  
Prentice Hall, 2017



# soa

Paradigmă de dezvoltare a software-ului  
care adoptă folosirea de servicii,  
oferind funcționalități solicitate de utilizatori

resursele sunt disponibile via o suită de servicii  
independente ale căror implementări  
nu trebuie să fie cunoscute (*black box*)

# soa

Componentele sistemului în ansamblu  
au un grad mare de **independență** (*de-coupling*)

serviciile trebuie proiectate să interacționeze  
fără a exista **dependențe** între acestea

# soa

Serviciile partajează un **contract formal**

necesitatea unei descrieri formal(izat)e a serviciului:  
operații oferite – interfața serviciului  
maniera de interschimb a datelor – cerere + răspuns  
maniera de descoperire (*service discoverability*)  
proprietăți specifice vizând utilizarea (SLA, SLO, SLI)

**soa**

Accept al utilizării serviciului

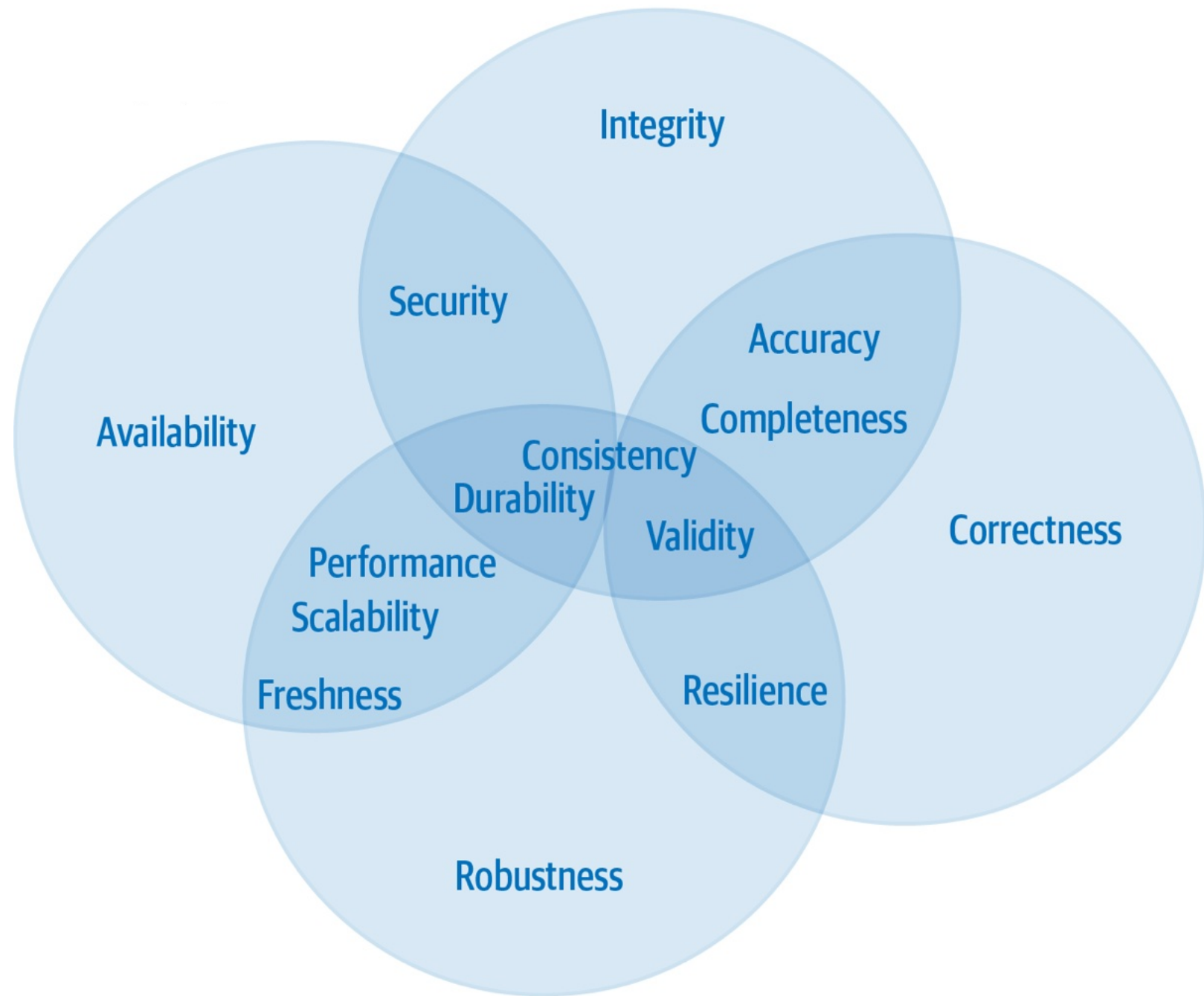
*SLA – Service-Level Agreement*

# soa

Scopuri (objective) – **SLO** (*Service Level Objectives*)  
*“proper level of reliability” targeted by the service*

îndeplinirea 100% a obiectivelor e imposibilă

A. Hidalgo, *Implementing Service Level Objectives*, O'Reilly, 2020  
studii de caz: [sre.google/workbook/slo-engineering-case-studies/](https://sre.google/workbook/slo-engineering-case-studies/)



proprietăți de interes: *availability, correctness, integrity, performance, resilience, robustness, scalability, security,...*

# soa

Indicatori – *SLI (Service Level Indicators)*

metrici sau (meta-)date reprezentând o proprietate specifică a unui serviciu – utilizate pentru a evalua SLO

exemple:

*uptime, average speed to answer, turn-around time, abandonment rate, mean time to recover, CPU usage,...*

# soa

Indicatori – *SLI (Service Level Indicators)*

*uptime* – perioada de timp în care un serviciu individual e operațional (rulează) pe o anumită platformă



**soa**

Indicatori – **SLI** (*Service Level Indicators*)

*availability* – perioada de timp în care un anumit serviciu răspunde efectiv la cererile formulate de clienți

# soa

Indicatori – **SLI** (*Service Level Indicators*)

*reliability* – perioada de timp în care un serviciu poate realiza operațiunile pe care pretinde că le poate face

context mai general: **SRE** (*Site Reliability Engineering*)

**sre.google**

arie de interes: *Web platform engineering*

# soa

Indicatori – **SLI** (*Service Level Indicators*)

pe baza monitorizării și jurnalizării activităților,  
se pot realiza diverse analize statistice via  
TSDB (*Time Series DataBase*) – a **metrics system**  
+  
*structured event database* – a **logging system**

# soa

Uzual, se implementează și un mecanism de alertare  
(*SLO alerting*)

pragul limită (*threshold alert*) de activare a unei alerte  
când se depășește valoarea unei metrici  
*e.g.*, memorie utilizată, număr de cereri/minut

**soa**

Serviciile pot fi recompuse/orchestrate  
conform cerințelor sau contextului de exploatare

# soa

Serviciile pot fi compuse din sub-servicii  
(*composable services*)  
ce pot fi reutilizate (*reusable services*)

serviciu compus – (macro-)serviciu



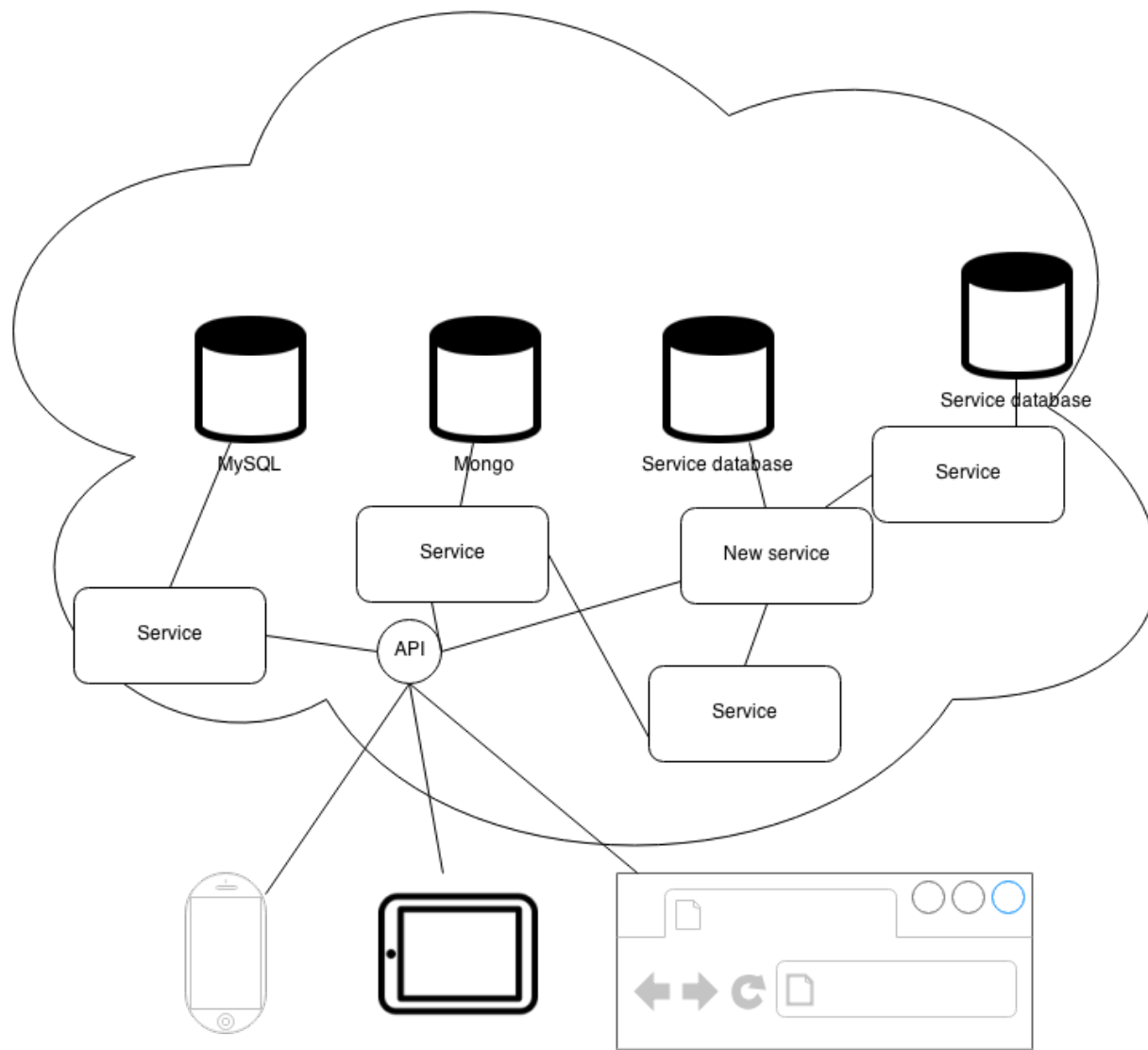
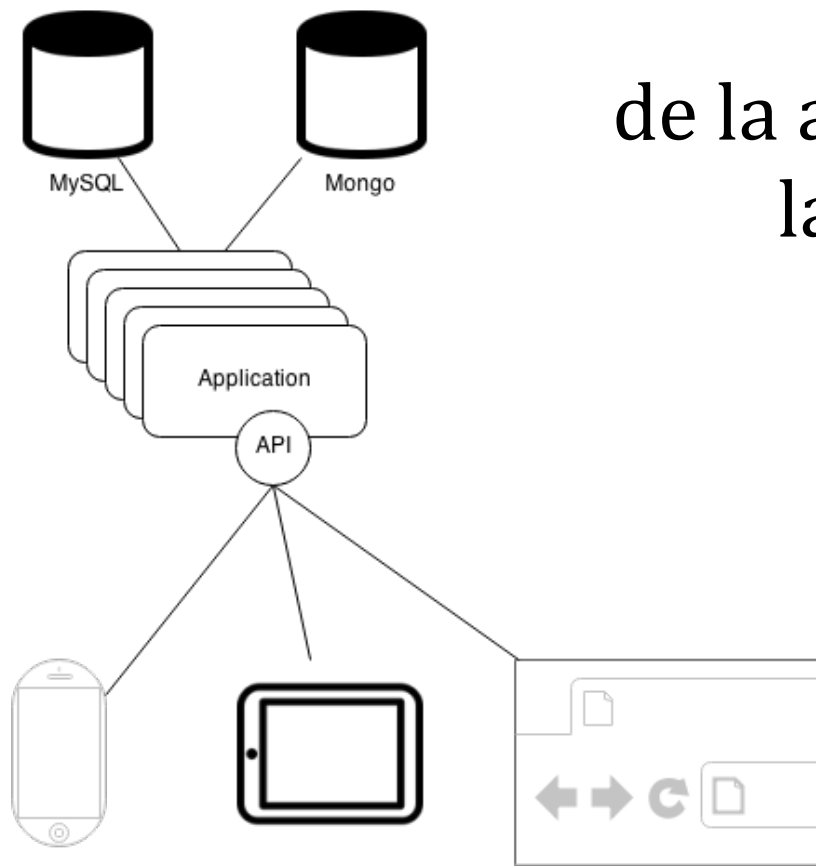
(micro-)servicii

**soa**

Serviciile nu vor depinde de starea comunicării  
(*statelessness*)

pentru a efectua o procesare, cantitatea de informație  
ce trebuie reținută trebuie să fie minimală

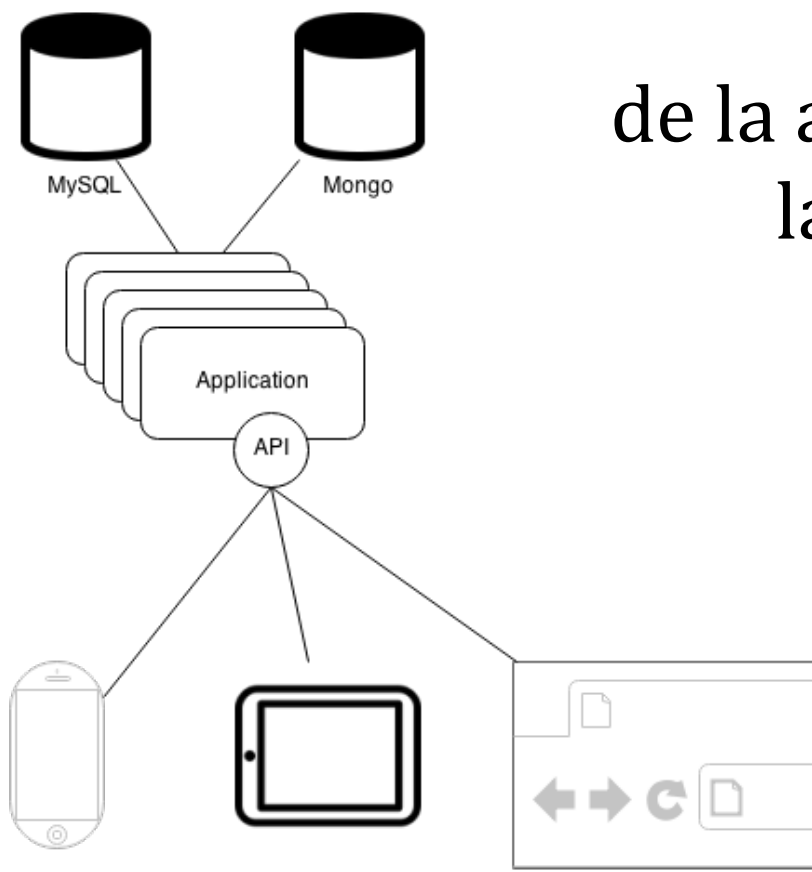
# de la aplicații Web dezvoltate tradițional la arhitecturi bazate pe servicii



Jay O'Connor (2014)

[tech.bellycard.com/blog/migrating-to-a-service-oriented-architecture-soa/](http://tech.bellycard.com/blog/migrating-to-a-service-oriented-architecture-soa/)



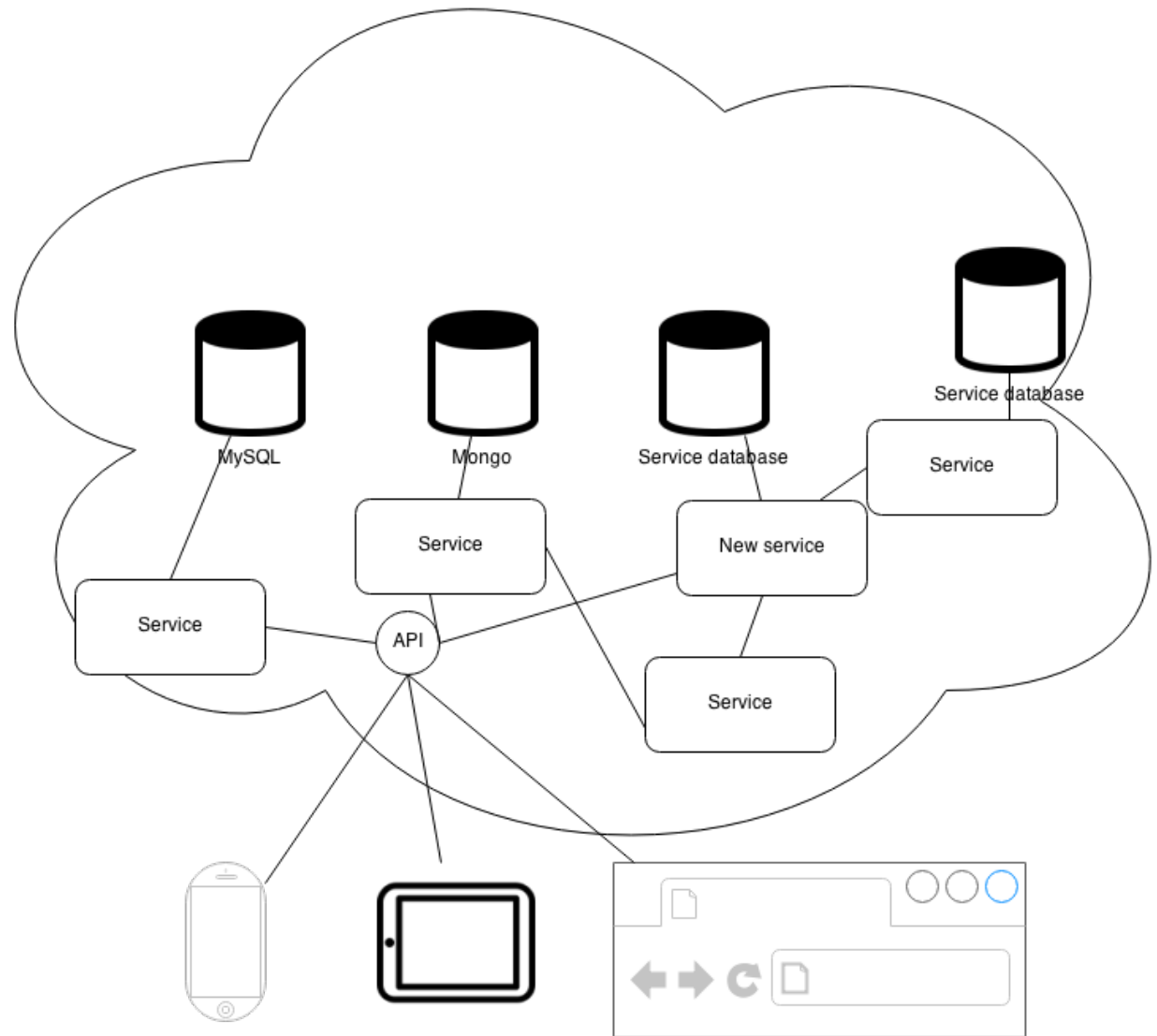


de la aplicații Web dezvoltate tradițional  
la arhitecturi bazate pe servicii

maniera clasică: aplicația Web expune funcționalități  
oferite posivilor săi clienți (navigatoare Web disponibile  
pe platforme multiple) pe baza datelor stocate persistent  
– uzual, o abordare arhitecturală MV\*

# de la aplicații Web dezvoltate tradițional la arhitecturi bazate pe servicii

SOA:  
fiecare funcționalitate  
este implementată  
de un serviciu  
independent  
ce poate recurge la  
alte servicii (interne)  
și la diverse sisteme  
de stocare a datelor



# soa

Serviciile – proprii și/sau externe –  
pot fi invocate la nivel de...

**server** (*back-end*)

și/sau

**client** (*front-end*)

...în manieră **sincronă** sau **asincronă**

# soa

## Orchestrare

existența unui serviciu principal (*service orchestrator*)  
care coordonează invocarea altor servicii și preluarea  
răspunsurilor oferite de acestea



punct de vedere centralizat

# soa

## Orchestrare

existența unui serviciu principal (*service orchestrator*)  
care coordonează invocarea altor servicii și preluarea  
răspunsurilor oferite de acestea

standard oficial (OASIS, 2007): **WS-BPEL**

(*Web Services Business Process Execution Language*)

[www.oasis-open.org/committees/tc\\_home.php?wg\\_abbrev=wsbpel](http://www.oasis-open.org/committees/tc_home.php?wg_abbrev=wsbpel)

# soa

## Coregrafie

descriere globală a serviciilor autonome participante la o activitate, definind maniera de interschimb de mesaje, reguli de interacțiune (*e.g.*, protoacoale) și contracte între două sau mai multe puncte-terminale



abordare descentralizată

# soa

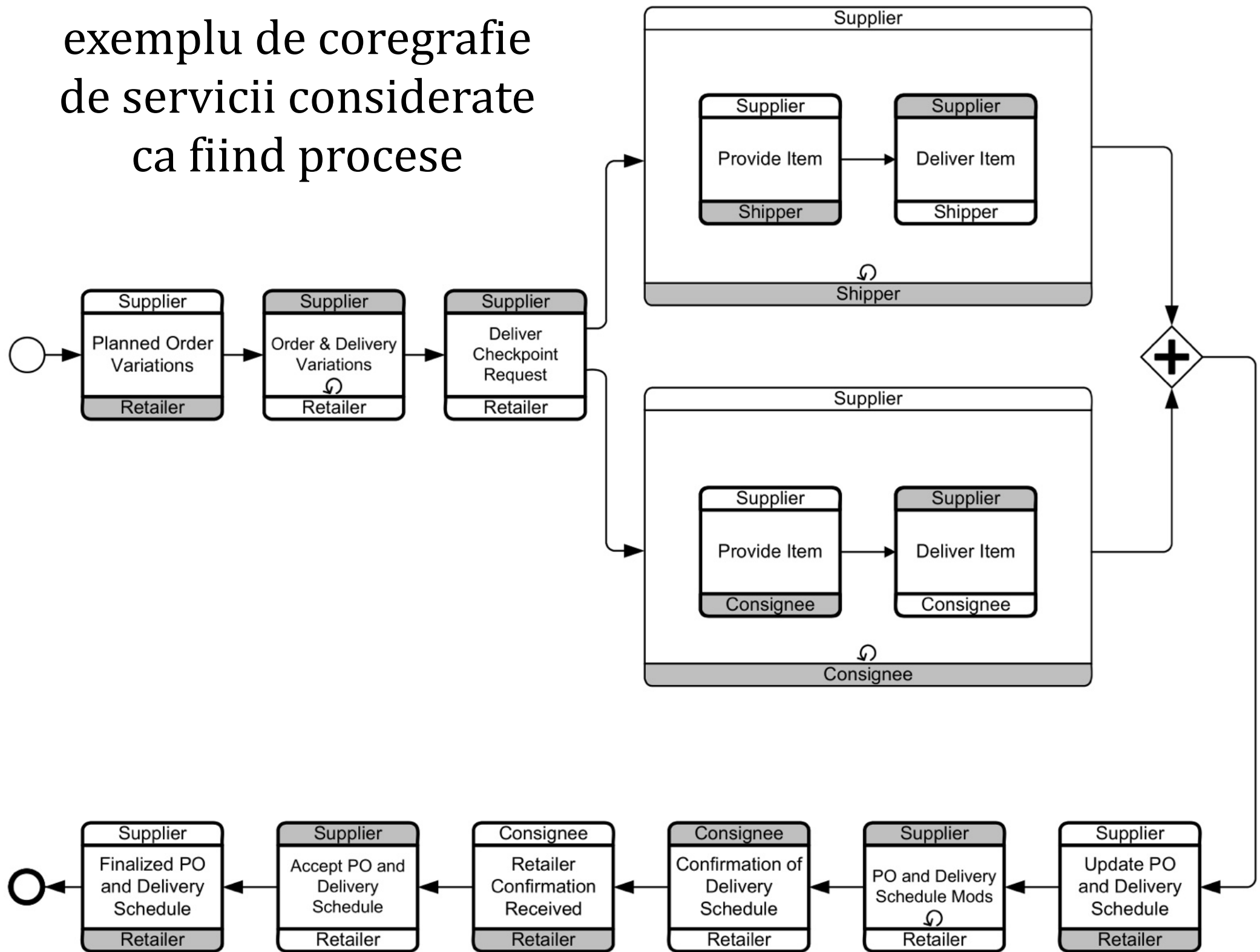
## Coregrafie

descriere globală a serviciilor autonome participante la o activitate, definind maniera de interschimb de mesaje, reguli de interacțiune (*e.g.*, protoacoale) și contracte între două sau mai multe puncte-terminale

*Web Services Choreography Description Language*

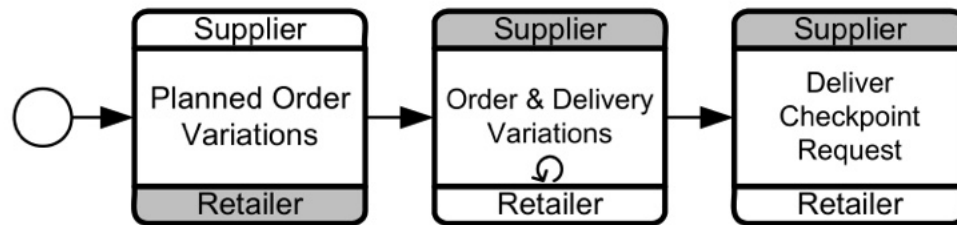
[www.w3.org/TR/ws-cdl-10-primer/](http://www.w3.org/TR/ws-cdl-10-primer/)

exemplu de coregrafie  
de servicii considerate  
ca fiind procese

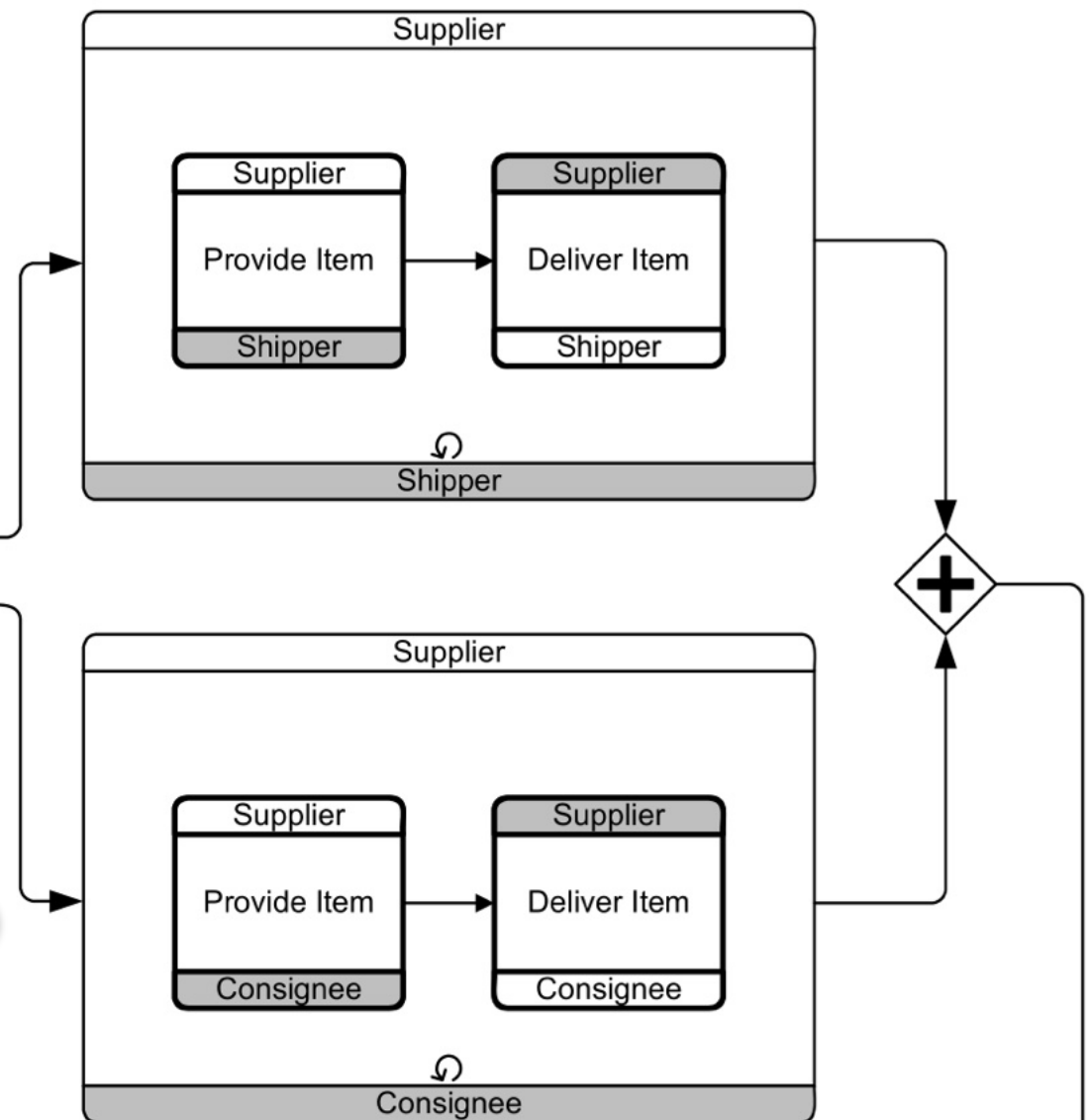




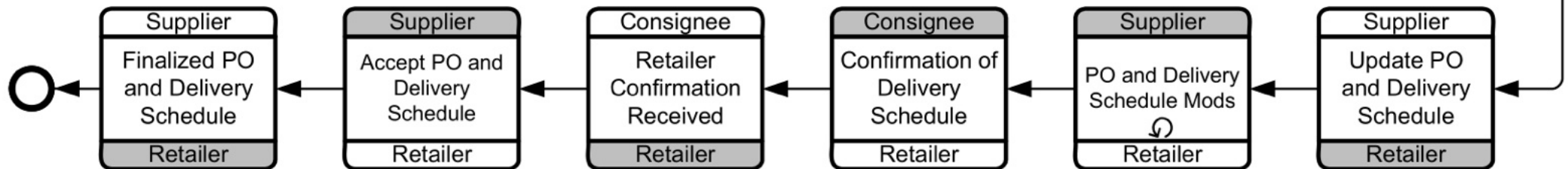
exemplu de coregrafie  
de servicii considerate  
ca fiind procese



aici, aprovizionarea  
cu mărfuri pentru  
un ofertant de produse



notația **BPMN** (*Business Process Model and Notation*)  
specificație OMG: [www.omg.org/spec/BPMN](http://www.omg.org/spec/BPMN)



 ofertant/utilizator de servicii Web

 API

micro-/macro-serv.

 server(e) *backend*

 mașină virtuală

 sist. tradiționale

 *workflow engine*

 servicii de bază (calendar, tranzacții,...)

 context al serviciului (cine, de ce,...)

 descrierea unui serviciu

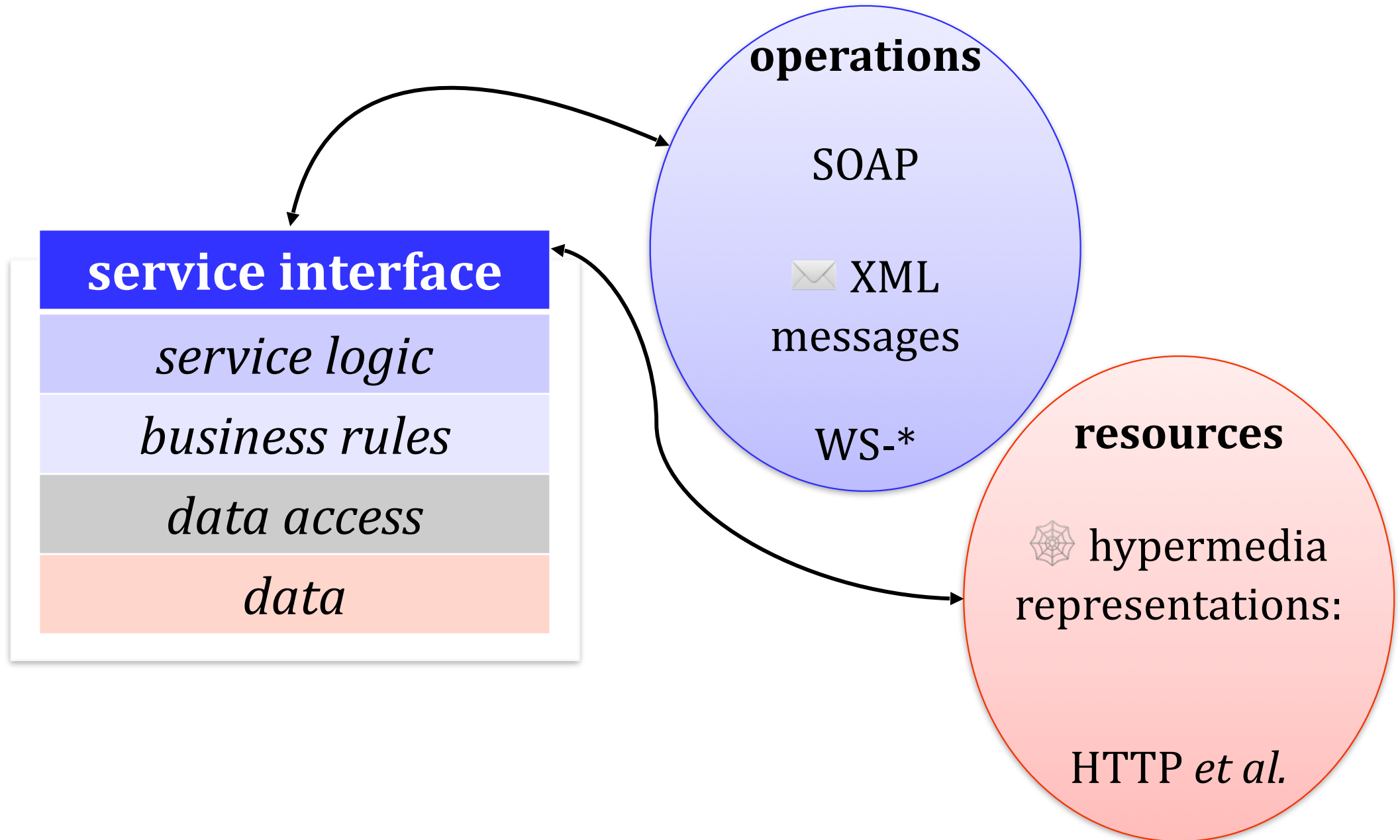
 servicii de regăsire

 protocoale: HTTP(S), WebSocket,...

structura conceptuală stratificată a unei platforme  
bazate pe servicii Web



Există o modalitate pragmatică privind dezvoltarea și invocarea serviciilor Web?



serviciile Web pot fi dezvoltate via **SOAP** și/sau **REST**

S. Tilkov, *REST: Not an Intro* (2013)

[speakerdeck.com/stilkov/rest-not-an-intro-1](http://speakerdeck.com/stilkov/rest-not-an-intro-1)

# rest: representational state transfer

Stil arhitectural de dezvoltare a aplicațiilor Web  
cu focalizare asupra reprezentării datelor

Roy Fielding

teză de doctorat, 2000 (*University of California, Irvine*)

[www.ics.uci.edu/~fielding/pubs/dissertation/rest\\_arch\\_style.htm](http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm)

**rest**

Rezultatul unei procesări conduce la obținerea  
unei **reprezentări** a unei resurse

# rest

Rezultatul unei procesări conduce la obținerea  
unei **reprezentări** a unei resurse

## resursă Web

utilizator având cont în cadrul unui sistem,  
mesaj al unei persoane, fotografie, flux de știri,  
componentă software, set de date (*dataset*), model 3D,...

# rest

Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

**reprezentare pe baza unui format de date**

textual sau binar

exemple tipice – formate deschise:  
HTML, JSON, CSV, PNG, SVG, PDF etc.



# rest

Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

**reprezentare pe baza unui format de date**

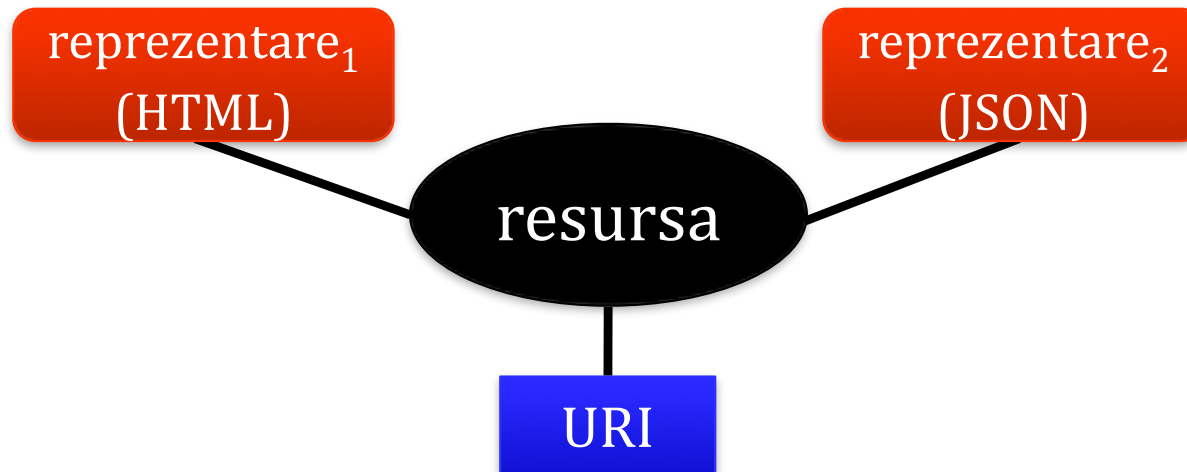
formatul reprezentării e desemnat de **tipuri MIME**  
**text/html, application/json, application/rdf+xml, image/webp**

detalii în N.Freed *et al.*, *Media Types*, 25 septembrie 2025  
[www.iana.org/assignments/media-types/media-types.xhtml](http://www.iana.org/assignments/media-types/media-types.xhtml)

# rest

Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

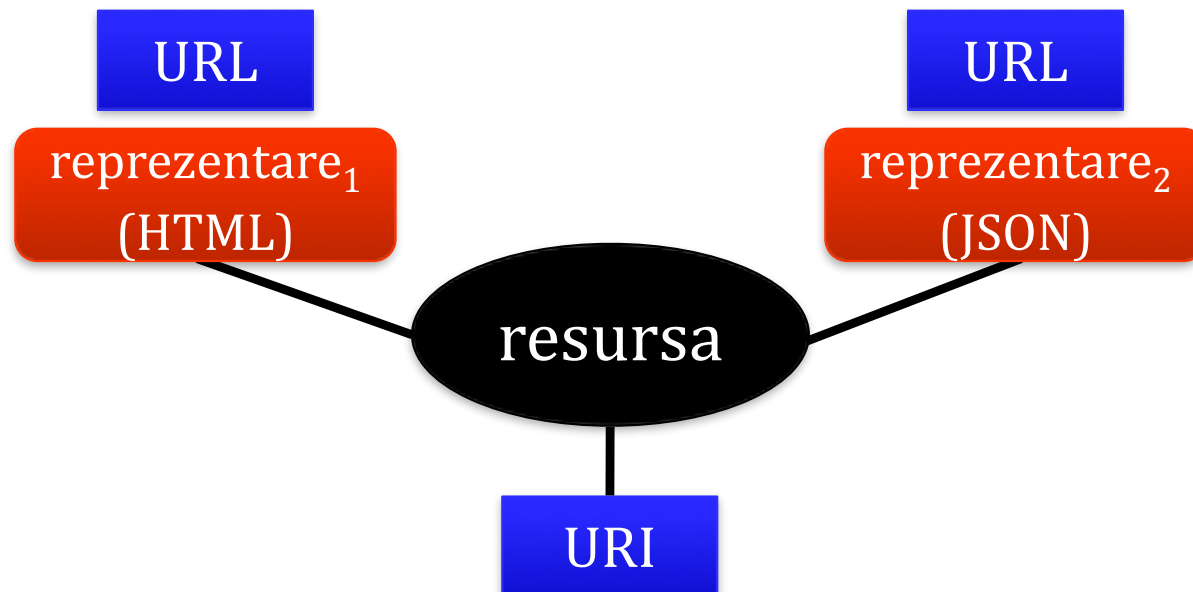
reprezentările aceleiași resurse  
– desemnate de un URI unic – pot fi multiple



# rest

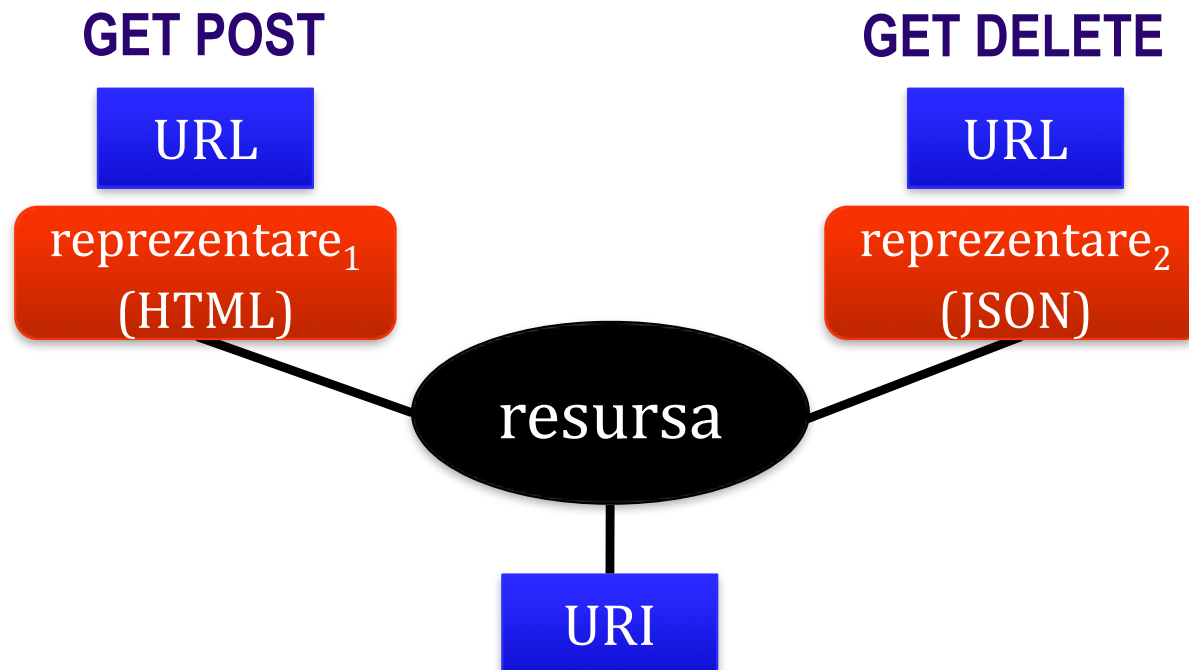
Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

fiecare reprezentare a unei resurse are asociat un URL



# rest

Clienții (*e.g.*, navigatoare Web, roboți, *player*-e etc.) interacționează cu reprezentările resurselor via **verbe** „accesează”: **GET**, „modifică”: **POST**, „șterge”: **DELETE**,...



**rest**

Verbele (acțiunile) sunt stipulate de un protocol

# rest

Verbele (acțiunile) sunt stipulate de un protocol

⚠️ remarcă:

deși uzual se recurge la HTTP,  
în unele cazuri se pot folosi alte protocoale

exemplu: Realtime API OpenAI adoptă WebSocket  
[platform.openai.com/docs/guides/realtime](https://platform.openai.com/docs/guides/realtime)

**rest**

## *HyperText Transfer Protocol*

protocol fiabil, de tip cerere/răspuns

port standard de acces: **80**

de (re)văzut prelegerea privitoare la HTTP, *cookie*-uri, sesiuni Web:  
[edu.info.uaic.ro/web-technologies/web-film.html#week2](http://edu.info.uaic.ro/web-technologies/web-film.html#week2)

# rest

## GET

accesează (preia) o reprezentare a unei resurse

nu conduce la modificarea stării serverului

– siguranță (*safe*)

idempotență (*idempotent*) – cereri identice vor conduce la oferirea aceluiași răspuns (aceeași reprezentare)



**rest**

**HEAD**

similară cu GET, dar furnizează doar meta-date  
(nu oferă reprezentarea propriu-zisă)

*e.g.*, ultima actualizare, lungimea conținutului,...

# rest

## PUT

înlocuiește (actualizează) o reprezentare de resursă sau  
eventual creează o resursă la nivel de server Web  
(al cărei URI e deja cunoscut)

uzual, returnează URI-ul resursei

nu e considerată *safe*, dar este idempotentă

**rest**

**PATCH**

permite actualizarea parțială a unei reprezentări  
a unei resurse (PUT nu oferă o asemenea facilitate)

nu este *safe* și nici idempotentă

**rest**

**POST**

creează o resursă (uzual, subordonată altei resurse)

opțional, pot fi realizate procesări suplimentare

nu este nici *safe*, nici idempotentă

clientul nu cunoaște *a-priori*  
care va fi URI-ul resursei ce va fi create

**rest**

**DELETE**

șterge (elimină) o resursă desemnată de un URI

este idempotentă

# rest

## OPTIONS

permite clientului să determine diverse cerințe privitoare la o resursă (*e.g.*, dacă o resursă poate fi ștearsă) sau facilitățile expuse de un server (de exemplu, suportul oferit de un *proxy*)

| <b>Metoda HTTP</b> | <b><i>Idempotent</i></b> | <b><i>Safe</i></b> |
|--------------------|--------------------------|--------------------|
| <b>GET</b>         | ✓                        | ✓                  |
| <b>POST</b>        | ✗                        | ✗                  |
| <b>PUT</b>         | ✓                        | ✗                  |
| <b>PATCH</b>       | ✗                        | ✗                  |
| <b>OPTIONS</b>     | ✓                        | ✓                  |
| <b>HEAD</b>        | ✓                        | ✓                  |
| <b>DELETE</b>      | ✓                        | ✗                  |

de parcurs și articolul Tamas Piroș,  
*RESTful API Design – POST vs PUT vs PATCH*, 2018  
[tpiros.dev/blog/restful-api-design-post-vs-put-vs-patch/](https://tpiros.dev/blog/restful-api-design-post-vs-put-vs-patch/)

**GET** /sparql?default-graph-uri=http%3A%2F%2Fdbpedia.org  
&query=SELECT+DISTINCT+%3Fclass%0D%0A  
WHERE+%7B%0D%0A++%3Fclass+a+owl%3AClass%0D%0A  
%7D+LIMIT+7&format=application%2Fsparql-results+json  
&timeout=30000 **HTTP/1.1**

URL  
encoding

**Host:** dbpedia.org

**User-Agent:** Mozilla/5.0 ... Gecko/20100101 Firefox/...

**Accept:** text/html,application/xhtml+xml,application/xml;q=0.9,\*/\*;q=0.8

**Accept-Language:** en,en-GB;q=0.5

**Accept-Encoding:** gzip, deflate, br, zstd

**DNT:** 1

**Priority:** u=0, i

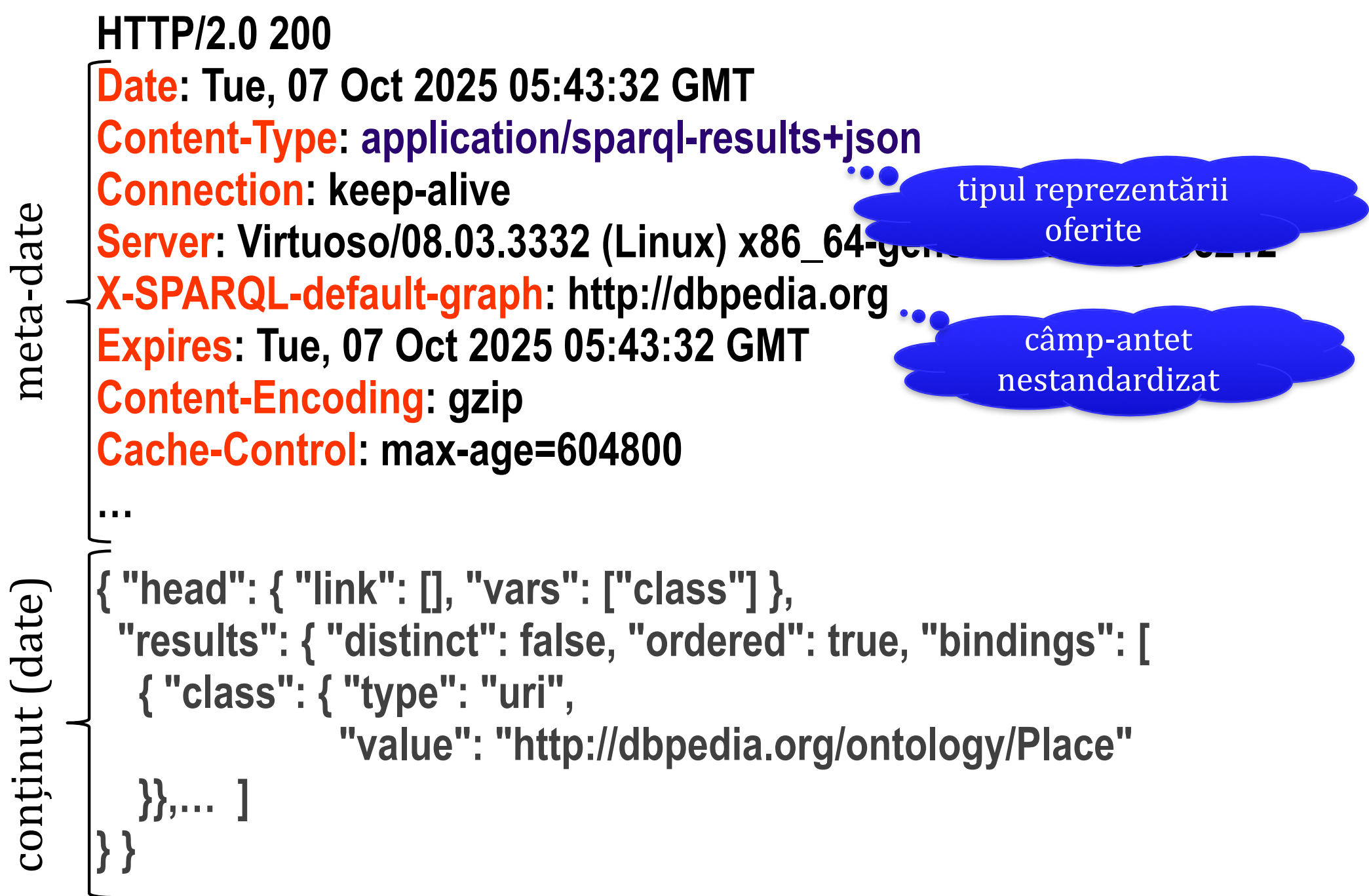
**Referer:** <http://dbpedia.org/sparql>

**Connection:** keep-alive

câmpuri-antet  
și valorile lor

un mesaj HTTP de tip cerere (*request*) trimis de client  
(uzual, un *browser* Web) spre server  
aici, o cerere încapsulând o interogare SPARQL către DBpedia

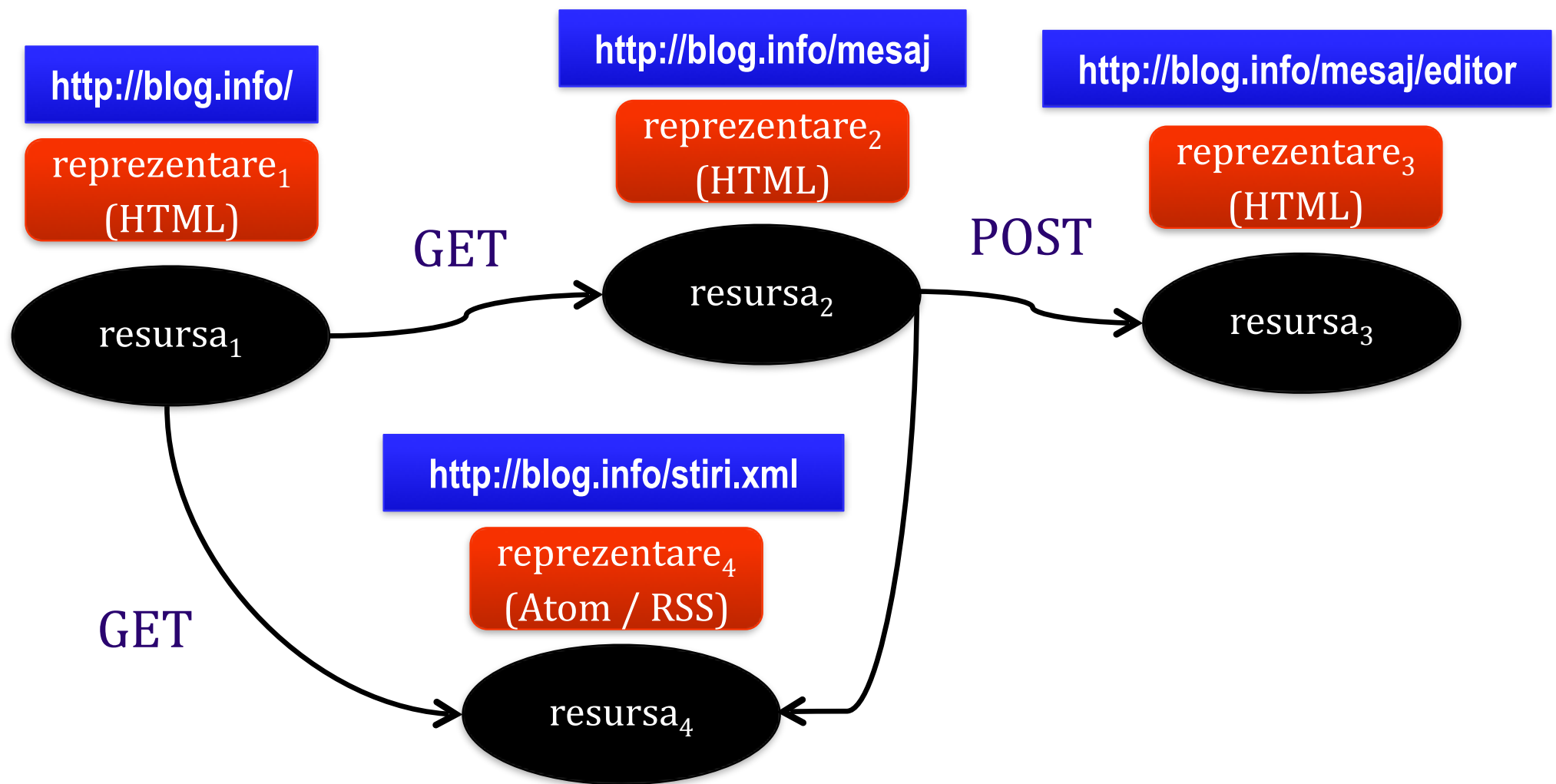




un mesaj HTTP de tip răspuns (*response*) de la server  
aici, în format JSON

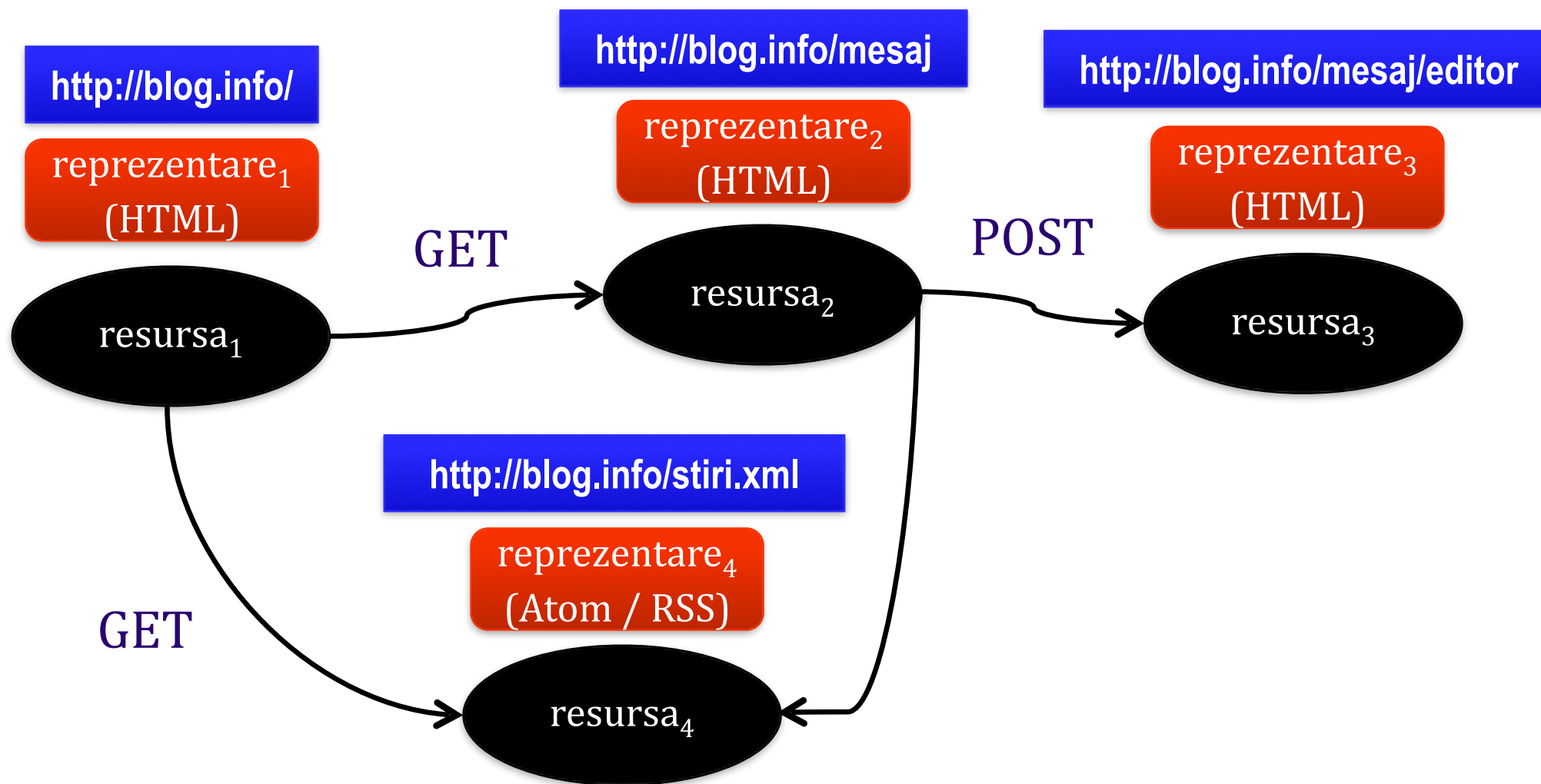
# rest

Orice accesare a unei reprezentări  
plasează aplicația – ori clientul Web – într-o **stare**  
ce va fi schimbată în urma unui **transfer** de date  
(accesarea altei reprezentări)



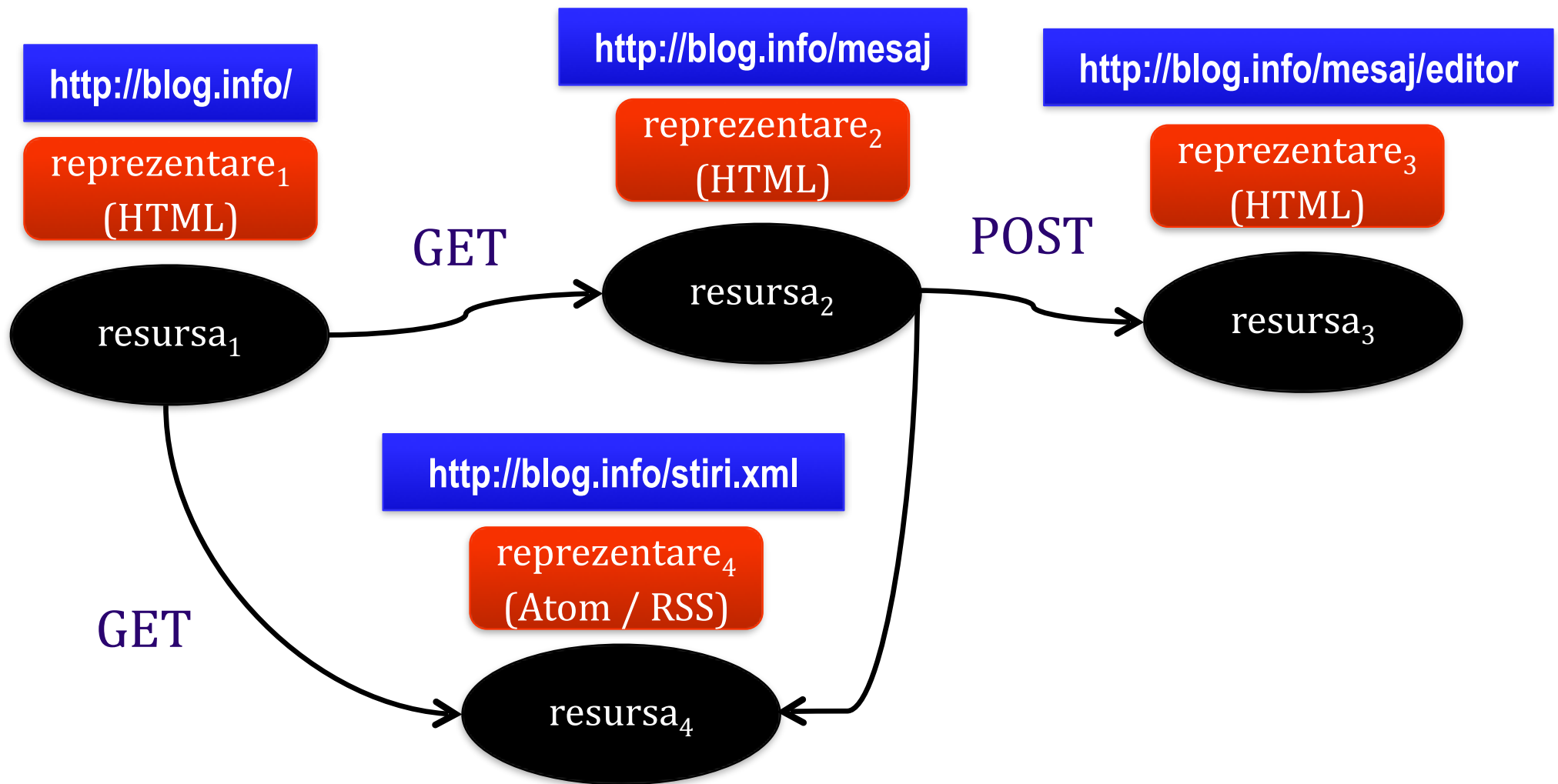
aplicație Web  $\equiv$  automat finit nedeterminist

tranzițiile dintre stări sunt dictate de transferuri de date  
pe baza metodelor protocolului – uzual, HTTP(S)



pe baza hipermediei – graful de legături între resurse –,  
dintr-o anumită stare (reprezentare)  
se pot realiza tranziții la alte stări

utilizatorul/programul accesează altă resursă via un URI



**HATEOAS** (*Hypermedia As The Engine Of Application State*)

[nordicapis.com/designing-a-true-rest-state-machine/](http://nordicapis.com/designing-a-true-rest-state-machine/)  
[nordicapis.com/rest-state-machine-revisited](http://nordicapis.com/rest-state-machine-revisited)

# rest

Transferul se realizează prin protocolul **HTTP**

Reprezentarea este modelată conform unui format

– *e.g.*, **JSON** sau **XML** – și  
indicată prin tipuri **MIME** (*media types*)

Adresabilitatea se rezolvă via **URI**

# rest

Aplicațiile care invocă funcționalități (servicii) consumă reprezentări de resurse – în stilul *pull*

# rest

Fiecare cerere este considerată independentă,  
fără a se lua în considerație contextul

*stateless server*



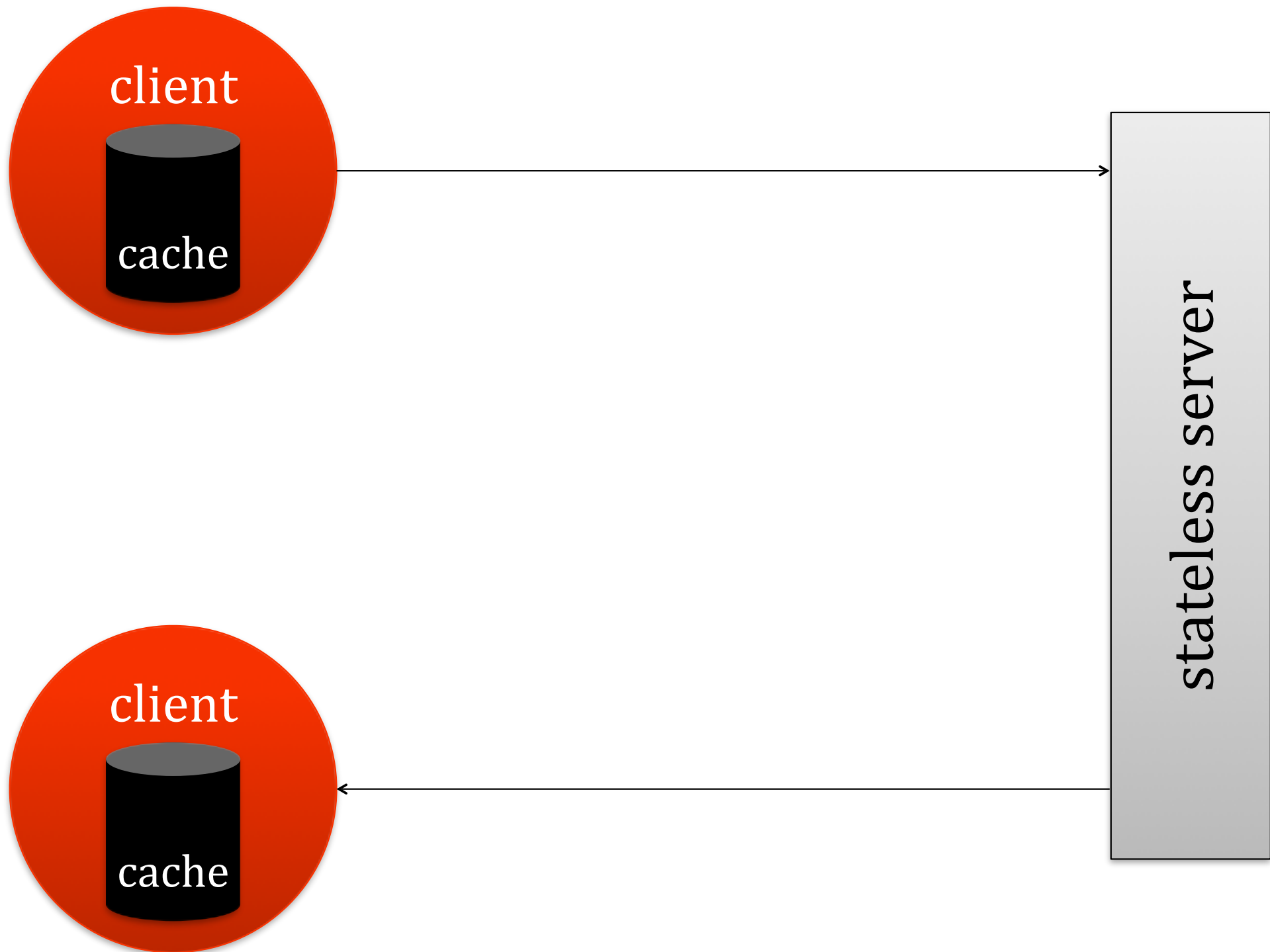
# rest

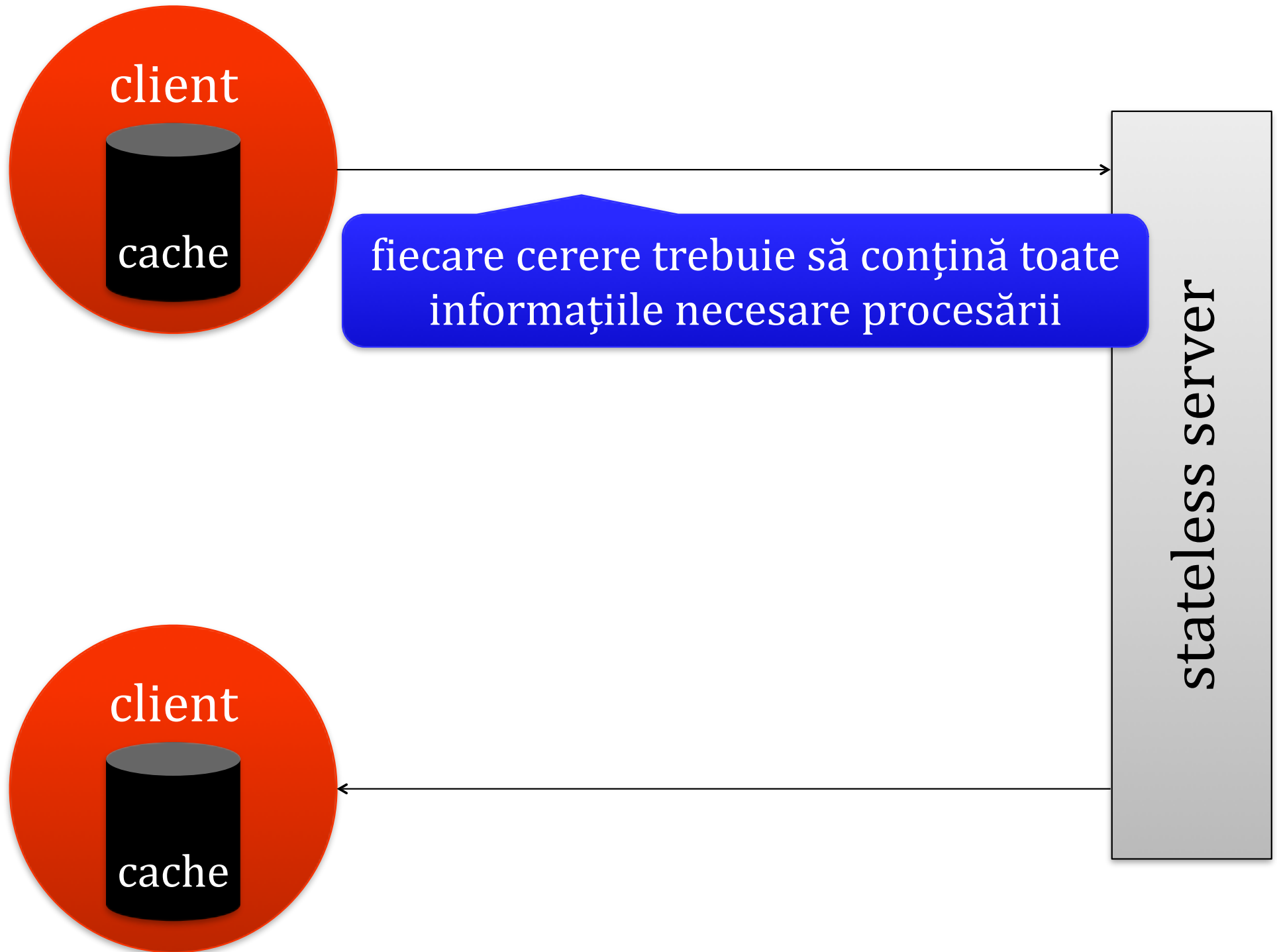
Reprezentările de resurse pot fi stocate temporar

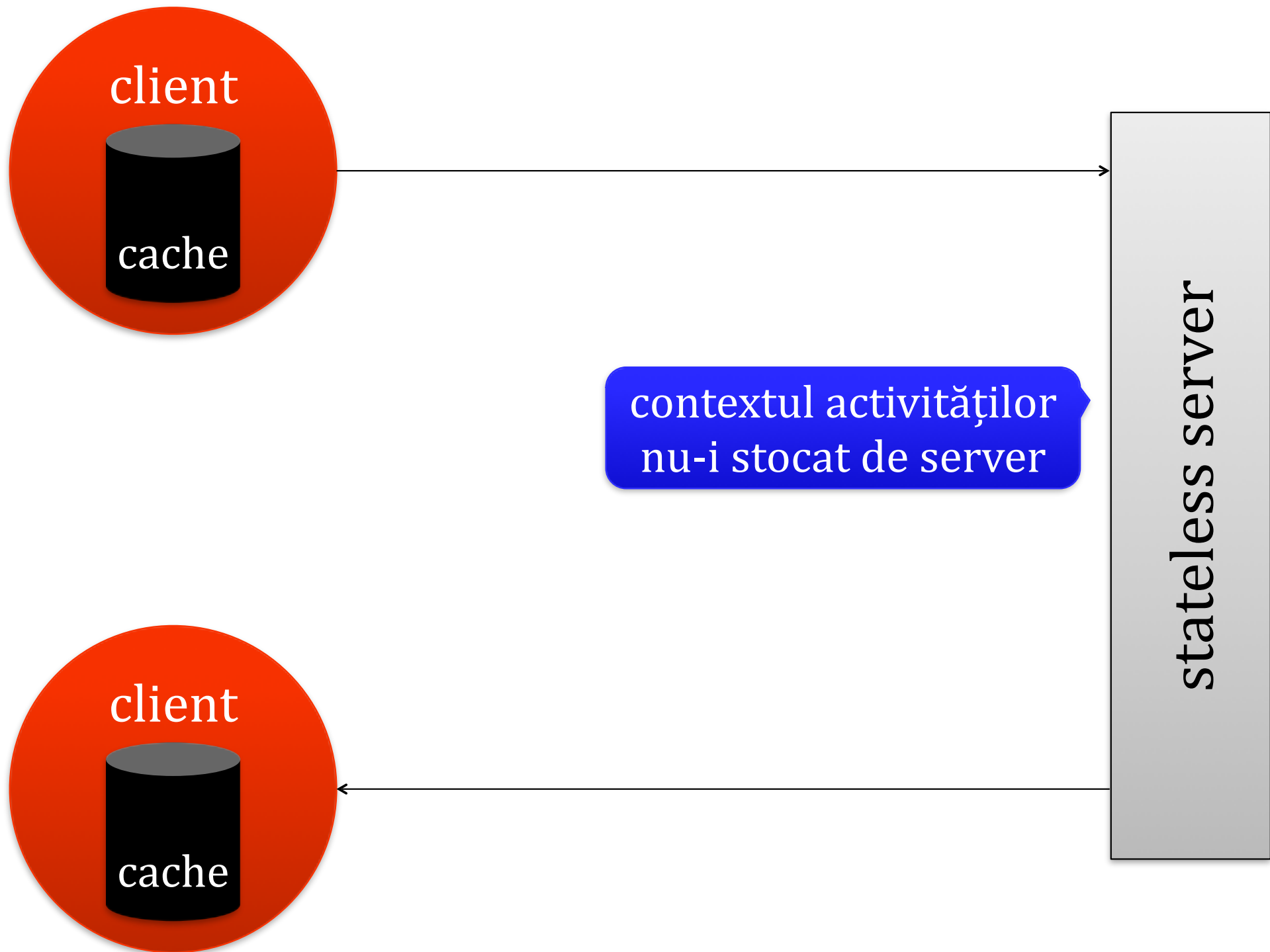
## *caching*

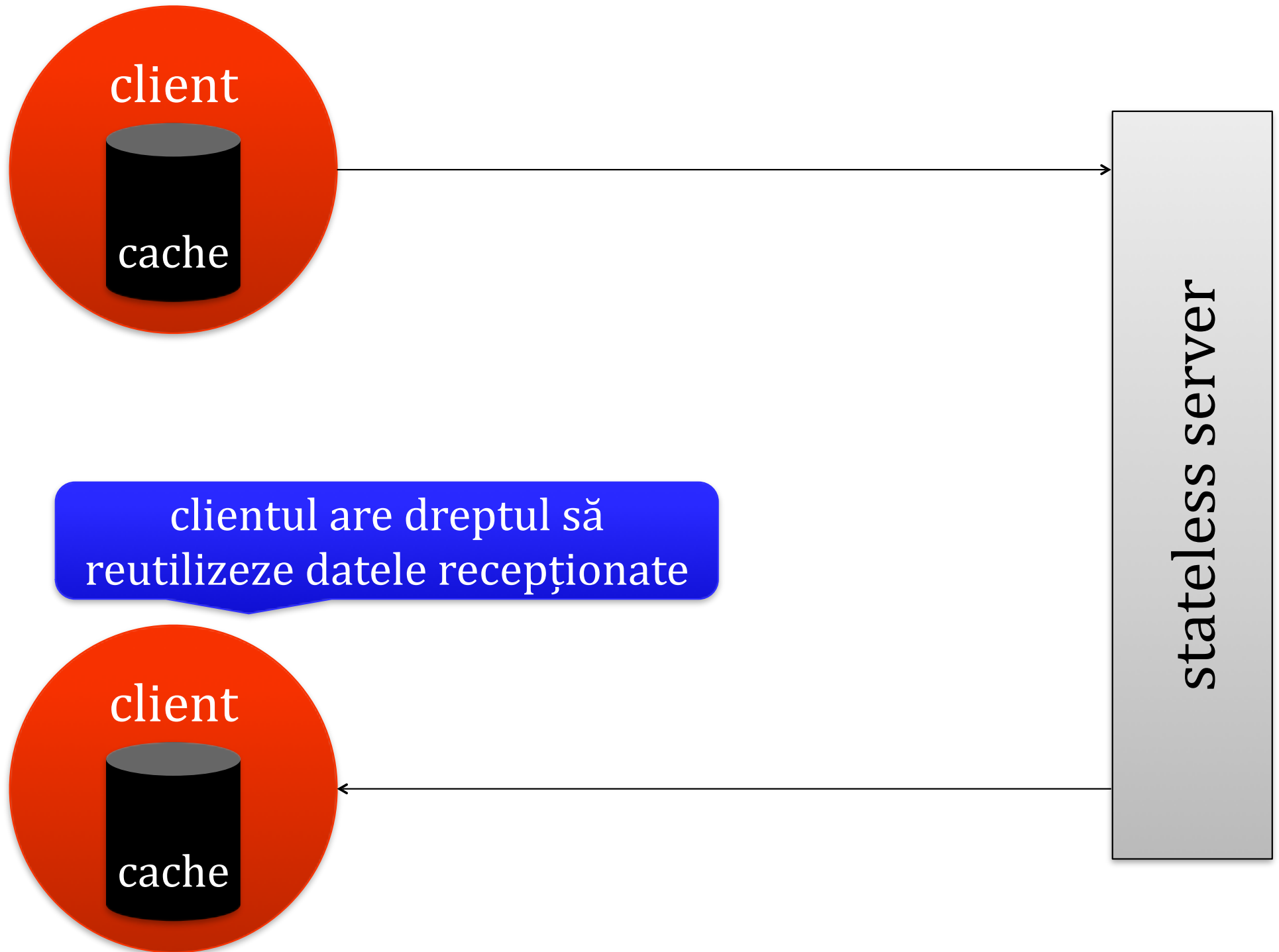
detalii de interes:

[developer.mozilla.org/docs/Web/HTTP/Caching](https://developer.mozilla.org/docs/Web/HTTP/Caching)





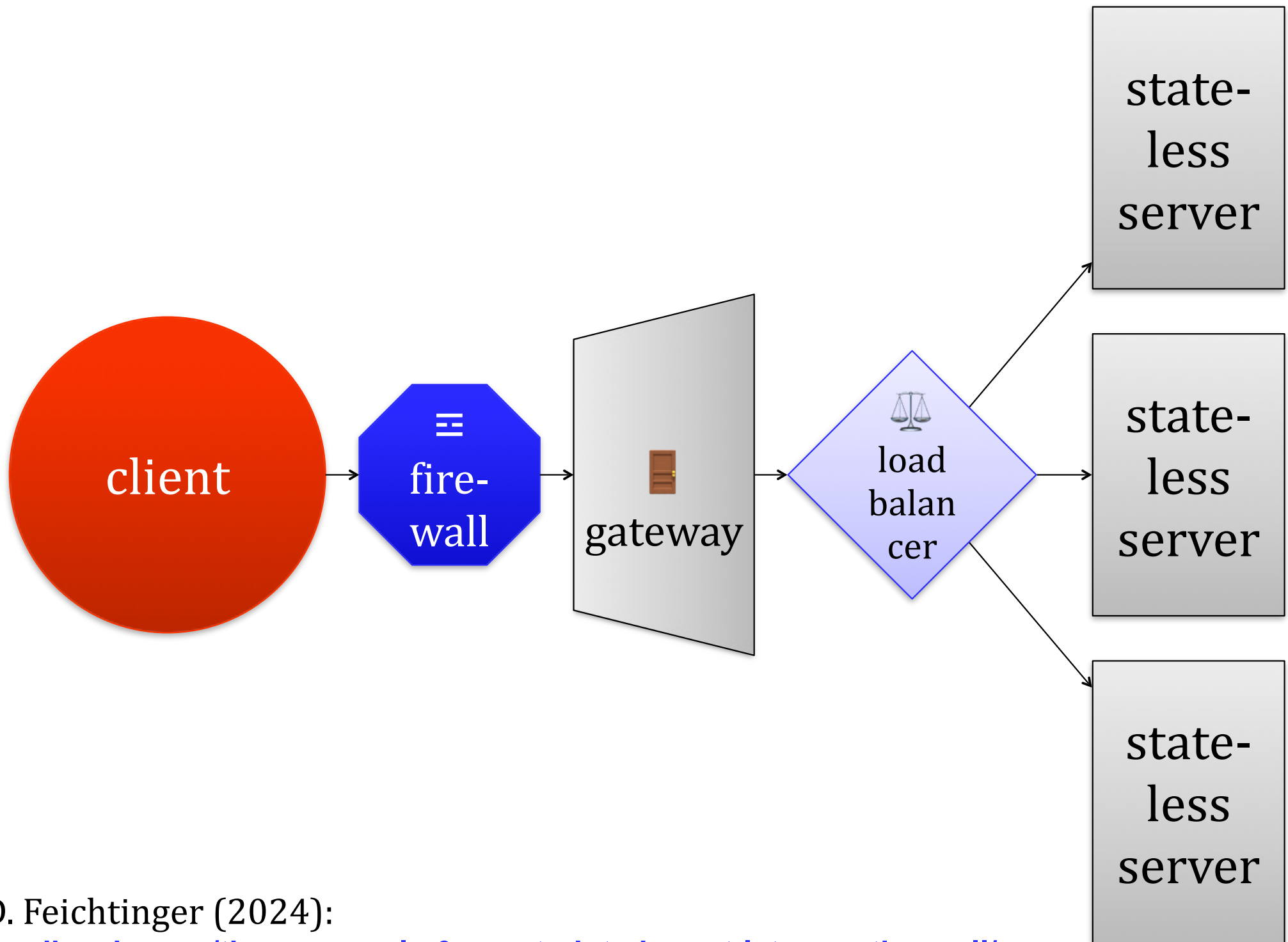




**rest**

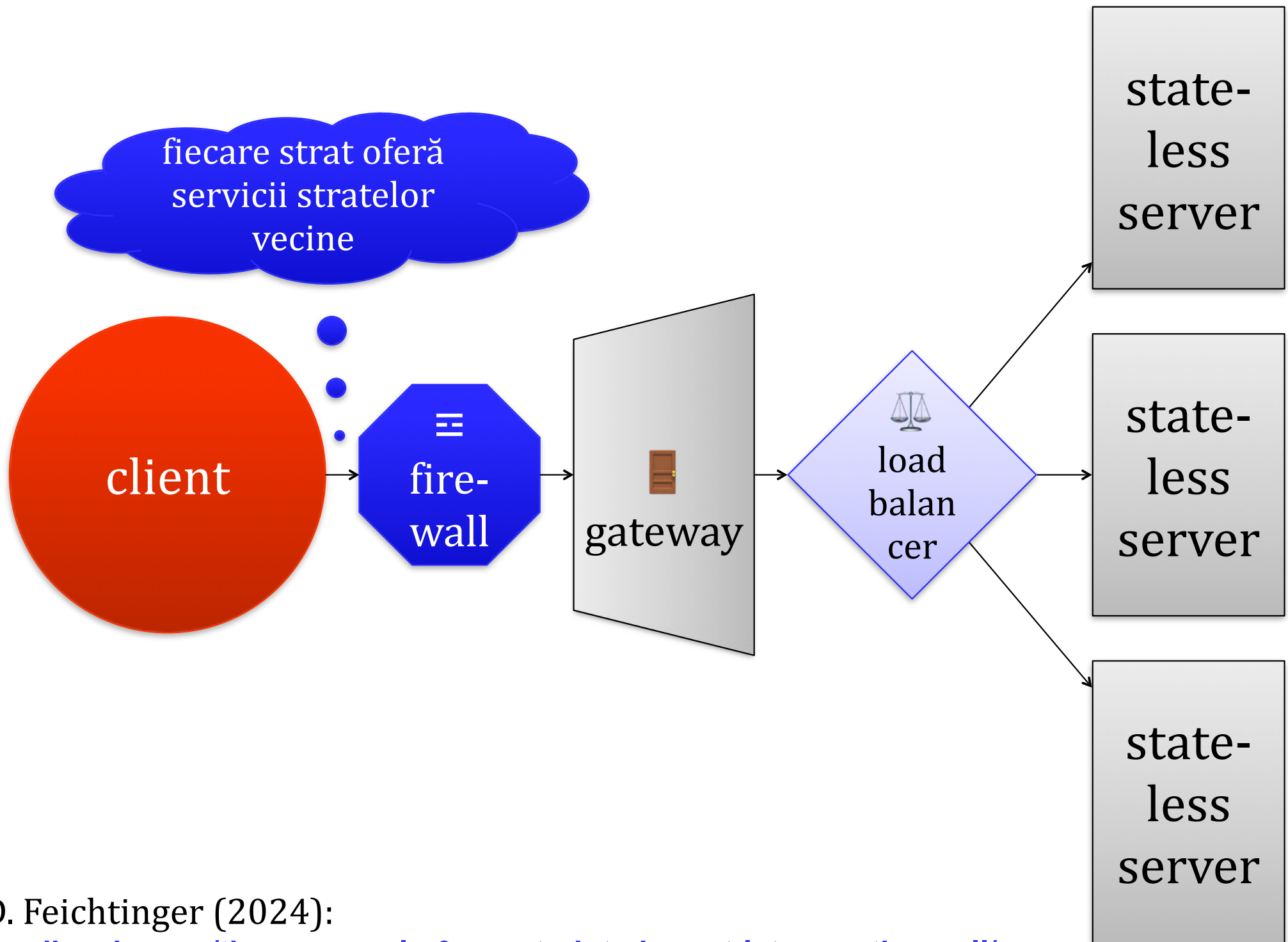
Aplicația Web dezvoltată va fi stratificată

*layered system*



D. Feichtinger (2024):

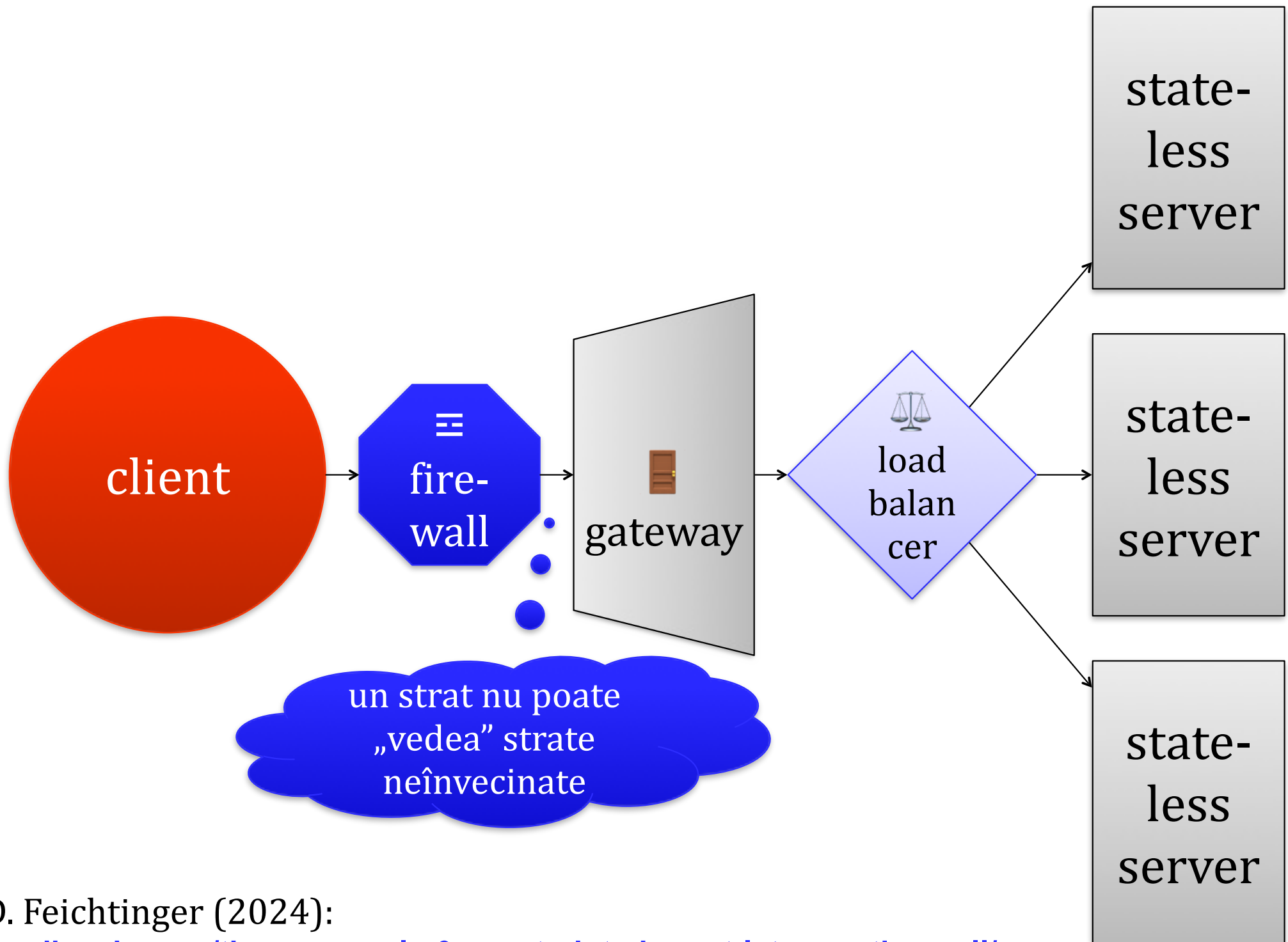
[nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/](https://nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/)



D. Feichtinger (2024):

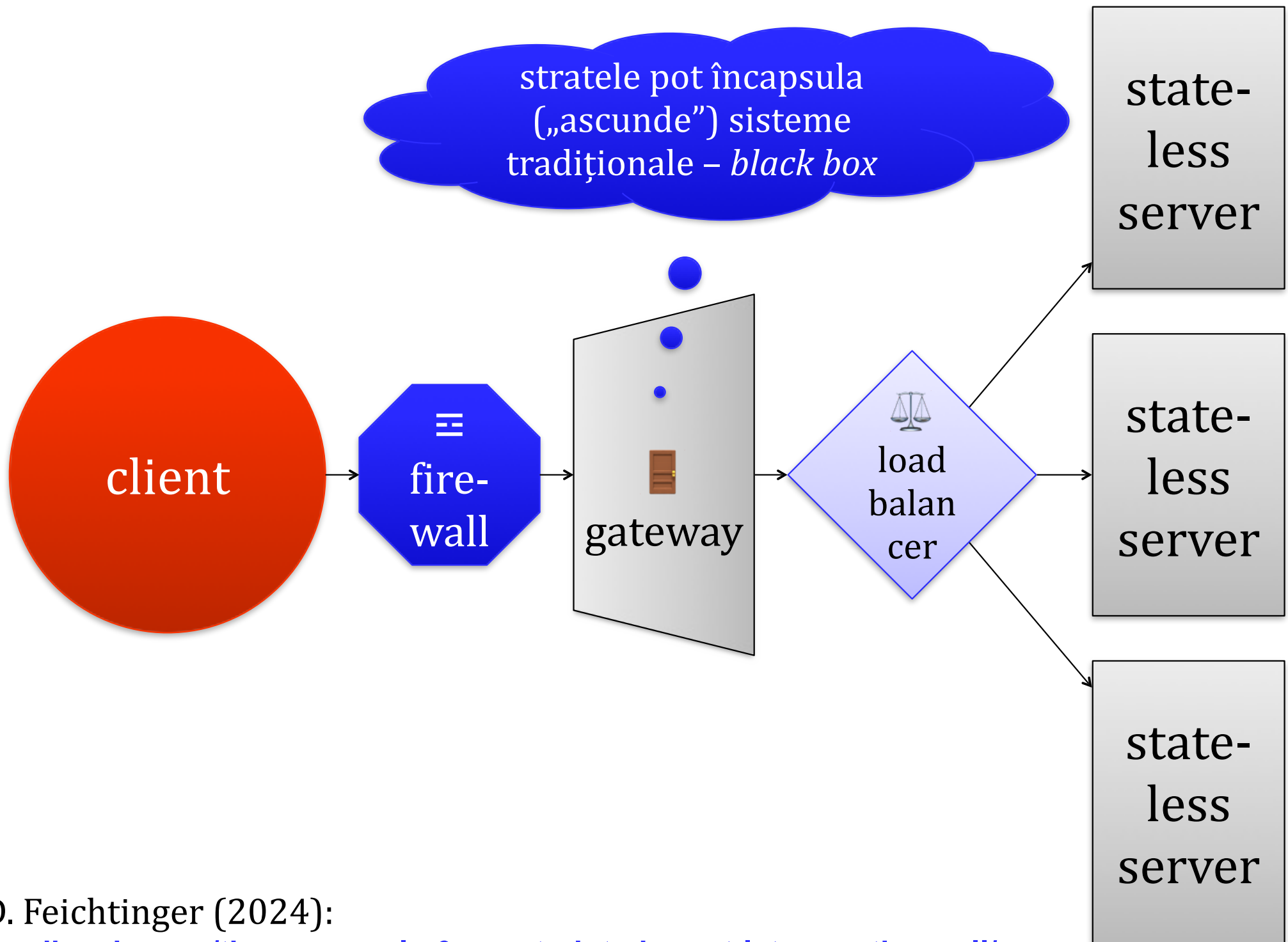
[nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/](https://nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/)





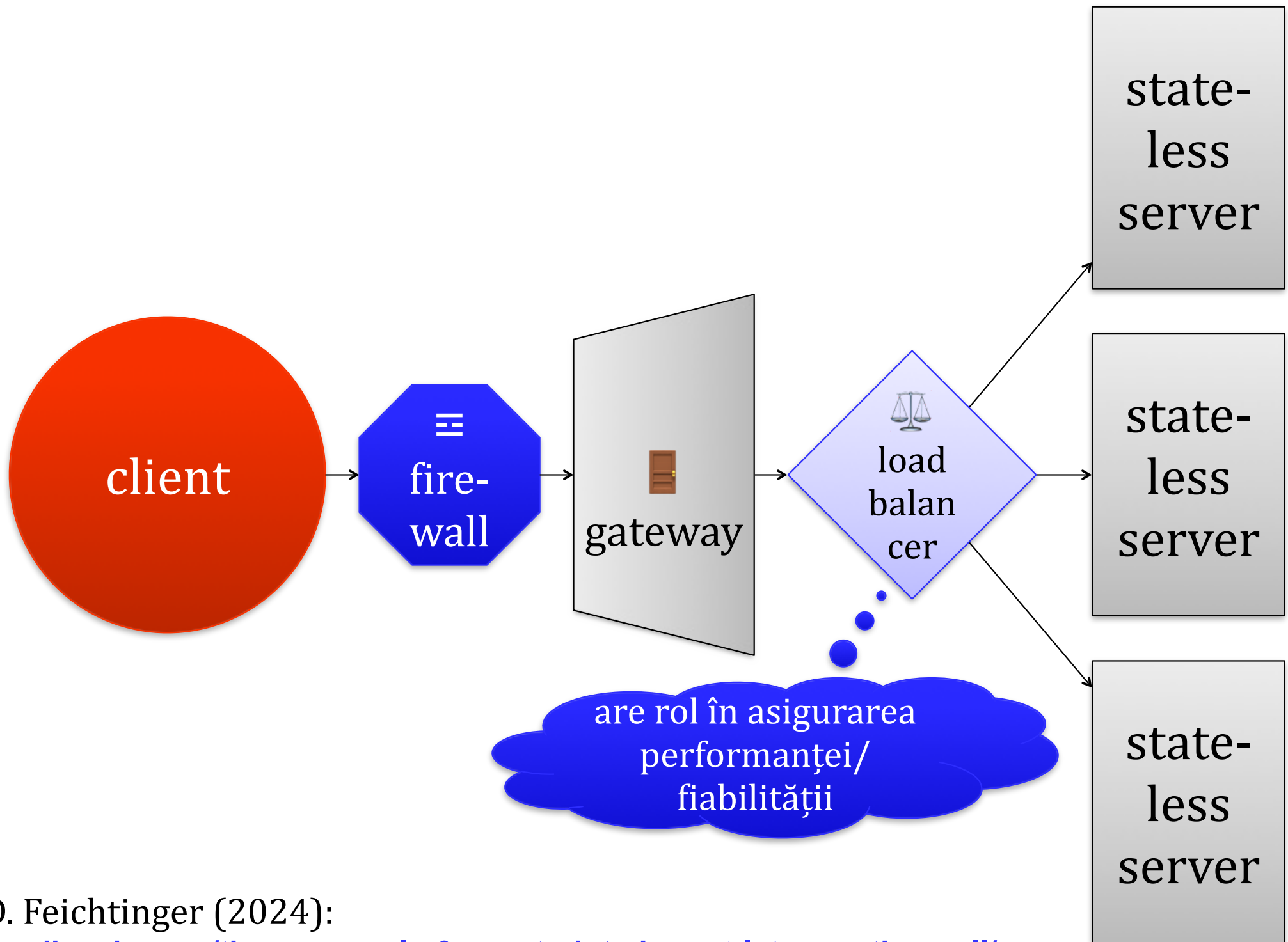
D. Feichtinger (2024):

[nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/](https://nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/)



D. Feichtinger (2024):

[nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/](https://nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/)



D. Feichtinger (2024):

[nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/](https://nordicapis.com/there-are-only-6-constraints-in-rest-lets-use-them-all/)

## rest: exemplu

Serviciu pentru managementul adreselor Web favorite (*bookmark-uri*), cu posibilitatea atașării de termeni de conținut (*tag-uri*) și comentarii

*social bookmarking*

abordări similare: Digg, Pinterest, Pocket, Reddit etc.

## rest: exemplu

Serviciu pentru managementul adreselor Web favorite (*bookmark*-uri), cu posibilitatea atașării de termeni de conținut (*tag*-uri) și comentarii

funcționalități vizând managementul *bookmark*-urilor:  
listare (eventual, cu filtrare după diverse criterii),  
adăugare, editare, ștergere, partajare

| Resursa                  | URL               | Metoda | Reprezentare             |
|--------------------------|-------------------|--------|--------------------------|
| <i>Bookmark</i>          | /bookmarks/{hash} | GET    | application/bookmark+xml |
| <i>Bookmark</i>          | /bookmarks/{hash} | PUT    | application/bookmark+xml |
| <i>Bookmark</i>          | /bookmarks/{hash} | DELETE |                          |
| Lista de adrese          | /bookmarks        | GET    | application/atom+xml     |
| Lista de utilizatori     | /users            | GET    | application/atom+xml     |
| Lista de <i>tag</i> -uri | /tags             | GET    | application/atom+xml     |
| Pagina principală        | /                 | GET    | application/xml          |

GET /bookmarks

200 OK

Content-type: application/atom+xml ..

răspuns XML (în format Atom)  
oferit de serviciu

<?xml version="1.0"?>

<feed xmlns="http://www.w3.org/2005/Atom">

<title>Bookmarks</title>

<entry>

<title>O resursă interesantă</title> ..

<link href="/bookmarks/a211528f...bdcf"/>

<summary>

http://undeva.info/o-resursa-interesanta

</summary>

</entry>

<!-- eventual, alte elemente <entry>... -->

</feed>

*digest – hash*  
(SHA-1, SHA-3,...)

obținerea  
bookmark-urilor

GET /bookmarks/a211528f...bdcf

200 OK

Content-type: application/bookmark+xml

<bookmark>

<title>O resursă interesantă</title>

<url>http://undeva.info/o-resursa-interesanta</url>

<user href="/users/tux">tux</user>

<tags>

<tag href="/tags/interesting">interesting</tag>

<tag href="/tags/penguin">penguin</tag>

</tags>

</bookmark>

preluarea unui *bookmark*:  
răspunsul XML oferit de serviciul Web



**POST** /bookmarks

**Content-type:** application/bookmark+xml

...

**201 Created**

**Location:** /bookmarks/a211528f...bdcf

crearea  
unui *bookmark*

**PUT** /bookmarks/a211528f...bdcf

**Content-type:** application/bookmark+xml

...

**200 OK**

înlocuirea  
unui *bookmark*

# rest

Resursele se denumesc folosind URI-uri (URL-uri)

Reprezentările sunt interconectate prin URL-uri

“**Hypermedia as the engine of application state** *is a REST constraint.*”

(Roy Fielding, 2014)

Pot exista intermediari (*e.g., proxy, cache, gateway*)  
între clienți și resurse ► performanță, securitate,...

Transferul de date poate fi și asincron – *Ajax et al.*

O resursă poate avea asociate reprezentări ce pot fi accesate/alterate via operații HTTP

**CRUD** – *Create, Retrieve, Update, Delete*

| <i>Operation</i>        | SQL    | HTTP           |
|-------------------------|--------|----------------|
| <i>Create</i>           | INSERT | PUT POST       |
| <i>Read (Retrieve)</i>  | SELECT | GET            |
| <i>Update (Modify)</i>  | UPDATE | PUT POST PATCH |
| <i>Delete (Destroy)</i> | DELETE | DELETE         |

# caz concret: *framework*-ul **LoopBack** – [loopback.io/doc/](https://loopback.io/doc/) operații↔REST↔model de implementare↔SQL

## Model create, retrieve, update, and delete operations

When you connect a model to a persistent data source such as a database, it becomes a *connected model* with a full set of create, read, update, and delete operations from the [PersistedModel](#) class:

| Operation        | REST                                                              | LoopBack model method (Node API)* | Corresponding SQL Operation |
|------------------|-------------------------------------------------------------------|-----------------------------------|-----------------------------|
| Create           | <a href="#">PUT /modelName</a><br><a href="#">POST /modelName</a> | <a href="#">create()</a>          | INSERT                      |
| Read (Retrieve)  | <a href="#">GET /modelName?filter=...</a>                         | <a href="#">find()</a>            | SELECT                      |
| Update (Modify)  | <a href="#">PUT /modelName</a>                                    | <a href="#">updateAll()</a>       | UPDATE                      |
| Delete (Destroy) | <a href="#">DELETE /modelName/<br/>/modelID</a>                   | <a href="#">destroyById()</a>     | DELETE                      |

(\*) Methods listed are just prominent examples; other methods may provide similar functionality; for example: `findById()`, `findOne()`, and `findOrCreate()`.

# rest

Interacțiunea cu un serviciu Web dezvoltat  
în stilul REST se poate face via un API  
(*Application Programming Interface*)



vezi prelegerea  
următoare



Putem adopta o metodologie vizând  
dezvoltarea de servicii Web (API-uri)  
aliniată paradigmei REST?

# rest: metodologie

Divizarea în resurse a setului de date  
ale problemei

clase tipice de resurse:

 **Utilizatori**

 **Documente** – alternative: **Fotografii, Produse, Software,...**

 **Metadata** – *e.g.*, **Comentarii, Formate, Locații, Platforme** etc.

# rest: metodologie

Divizarea datelor problemei în categorii de resurse

cazuri concrete:

*Discogs* – [www.discogs.com/developers/](http://www.discogs.com/developers/)

Artists Releases Labels Versions Orders Stats

*StackExchange* – [api.stackexchange.com](http://api.stackexchange.com)

Users Answers Badges Comments Questions Revisions Tags

*Unsplash* – [unsplash.com/documentation](http://unsplash.com/documentation)

Users Photos Collections Topics Stats



# rest: metodologie

„Numirea” prin URI a fiecărei resurse

exemplificări:

[web.info/Utilizatori/tux](#)

[web.info/Documente/pinguini-cu-mere-albastre](#)

# rest: metodologie

„Numirea” prin URI a fiecărei resurse

cazuri concrete:

accesarea știrilor referitoare la un subiect de interes

[www.reddit.com/r/programming/](http://www.reddit.com/r/programming/)

acces la prezentările SlideShare ale utilizatorului **busaco**

[www.slideshare.net/busaco/presentations](http://www.slideshare.net/busaco/presentations)

obținerea listei celor ce urmăresc un utilizator autentificat

[x.com/followers](http://x.com/followers)

# rest: metodologie

## Organizarea resurselor

aceste resurse (*object instances*) pot fi organizate  
în colecții (*collections*) sau depozite (*stores*)

D. Denicola, *Creating Truly RESTful APIs* (2013)

[www.slideshare.net/slideshow/creating-truly-res-tful-apis/17784394](http://www.slideshare.net/slideshow/creating-truly-res-tful-apis/17784394)

# rest: metodologie

## Organizarea resurselor

### colecție

catalog de resurse gestionate de **server**  
clienții pot propune alterarea colecției  
serverul decide care-i rezultatul unei operații

exemple (GitHub): [/orgs/openstack/repos](#), [/orgs/openstack/events](#)

# rest: metodologie

## Organizarea resurselor

### depozit

„rezervă” de resurse gestionată de **client**  
(inclusiv filtrare, sortare, paginare, accesare meta-date,...)

exemplu (GitHub): [/users/openstack/repos?page=2&per\\_page=3](/users/openstack/repos?page=2&per_page=3)

# rest: metodologie

Proiectarea reprezentării(lor) *acceptate*  
ce pot fi trimise de aplicația client  
și reprezentării(lor) *întoarse* spre client

de considerat formatele standard, comune:

**CSV** – *Comma Separated Values*

**JSON**(-LD) – *JavaScript Object Notation (-Linked Data)*

**XML** – *Extensible Markup Language*

# rest: metodologie

Proiectarea reprezentării(lor) *acceptate*  
ce pot fi trimise de aplicația client  
și reprezentării(lor) *întoarse* spre client

exemplu: date JSON solicitate serviciului via HTTP(S)

cerere a clientului – **Accept:** application/json

răspuns de la server – **Content-Type:** application/json

# rest: metodologie

Integrarea resurselor  
via legături hipertext + formulare

exemplificare (GitHub):

*“All resources may have one or more \*\_url properties linking to other resources. These are meant to provide explicit URLs so that proper API clients don’t need to construct URLs on their own.”*

[docs.github.com/en/rest#hypermedia](https://docs.github.com/en/rest#hypermedia)



# rest: metodologie

## Crearea de studii de caz

specificarea condițiilor de eroare și/sau de excepție

eventual, aspecte privind controlul versiunilor API-ului



## Users

[Show/Hide](#)[List Operations](#)[Expand Operations](#)[Raw](#)

**POST** /Users/login Login a user with username/email and password

**POST** /Users/logout Logout a user with access token

**GET** /Users/confirm Confirm a user registration with email verification token

**POST** /Users/reset Reset password for a user with email

**GET** /Users/{id}/accessTokens/{fk} Find a related item by id for accessTokens

**DELETE** /Users/{id}/accessTokens/{fk}

**PUT** /Users/{id}/accessTokens/{fk}

**GET** /Users/{id}/accessTokens

**POST** /Users/{id}/accessTokens Creates a new instance in accessTokens of this model.

**DELETE** /Users/{id}/accessTokens Deletes all accessTokens of this model.

**GET** /Users/{id}/accessTokens/count Counts accessTokens of User.

**POST** /Users Create a new instance of the model and persist it into the data source

**PUT** /Users Update an existing model instance or insert a new one into the data source

**GET** /Users Find all instances of the model matched by filter from the data source

**GET** /Users/{id}/exists Check whether a model instance exists in the data source

**HEAD** /Users/{id} Check whether a model instance exists in the data source

**GET** /Users/{id} Find a model instance by id from the data source

**DELETE** /Users/{id} Delete a model instance by id from the data source

**PUT** /Users/{id} Update attributes for a model instance and persist it into the data source

**StrongLoop API (IBM)**  
operații cu resurse specifice – aici **Users**

POST

/Users

Create a new instance of the model and persist it into the data source

PUT

/Users

Update an existing model instance or insert a new one into the data source

## Response Class

Model | Model Schema

```
{
  "realm": "",
  "username": "",
  "credentials": "object",
  "challenges": "object",
  "email": "",
  "emailVerified": false,
  "verificationToken": "",
  "status": "",
  "created": "",
  "lastUpdated": "".
```

# StrongLoop API

## testarea interactivă a API-ului

Response Content Type 

## Parameters

| Parameter | Value                                                                                                                                                                      | Description         | Parameter Type | Data Type                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data      | <div><div></div><div>Parameter content type:<br/><div><div>application/json</div><div>application/json</div><div>application/x-www-form-urlencoded</div></div></div></div> | Model instance data | body           | <div>Model   Model Schema</div> <div>User {<br/>  realm (string, optional),<br/>  username (string, optional),<br/>  credentials (object, optional),<br/>  challenges (object, optional),<br/>  email (string),<br/>  emailVerified (boolean, optional),<br/>  verificationToken (string, optional),<br/>  status (string, optional),<br/>  created (string, optional),<br/>  lastUpdated (string, optional),<br/>  id (number, optional)<br/>}</div> |

Try it out!



Aspecte practice de interes  
pentru dezvoltatori?

## rest: **privire pragmatică**

URL-urile desemnând resurse (concepte)  
de interes trebuie să fie simple și intuitive

utilizarea substantivelor pentru fiecare „lucru” (entitate)

colecții de resurse (uzual, la plural)  
**/students**

identificatori unici pentru membrii unei colecții  
**/students/tuxy** (concret) vs. **/students/69** (abstract)

## rest: **privire pragmatică**

URL-urile desemnând resurse (concepte)  
de interes trebuie să fie simple și intuitive

structura ierarhică a URL-urilor reprezintă ierarhia  
resurselor din cadrul domeniului modelat

exemplu (Discogs):

`/users/{name}/collection/folders/{f_id}/releases/{r_id}/instances/{i_id}`

# rest: privire pragmatică

Utilizarea verbelor (metodelor) HTTP pentru efectuarea de operații asupra unor (colecții de) resurse

| resursa<br>(URI)                                 | POST<br>(creează)         | GET<br>(accesează)              | PUT<br>(actualizează)                          | DELETE<br>(șterge)               |
|--------------------------------------------------|---------------------------|---------------------------------|------------------------------------------------|----------------------------------|
| <b>/students</b>                                 | creează<br>un student nou | listează studenții<br>existenți | actualizează<br>un set de studenți             | șterge toți<br>studenții         |
| <b>/students/69</b><br>(un URL deja<br>existent) | eroare 🙄                  | oferă date<br>despre student    | dacă există,<br>actualizează,<br>altfel eroare | șterge<br>studentul<br>respectiv |

# rest: **privire pragmatică**

Folosirea codurilor de stare HTTP  
[developer.mozilla.org/docs/Web/HTTP/Status](https://developer.mozilla.org/docs/Web/HTTP/Status)

exemple tipice:

*101 Switching Protocols*

*200 OK, 204 No Content, 206 Partial Content*

*303 See Other, 304 Not Modified*

*400 Bad Request, 401 Unauthorized, 403 Forbidden,*

*404 Not Found, 405 Method Not Allowed*

*500 Internal Server Error, 503 Service Unavailable*



# Error 502

Ray ID: 6eb1499bb939229 • 2022-03-14 06:51:22 UTC

Bad gateway



You

Browser  
Working



Frankfurt

Cloudflare  
Working



emojipedia.org

Host  
Error

## 451: Unavailable due to legal reasons

We recognize you are attempting to access this website from a country belonging to the European Economic Area (EEA) including the EU which enforces the [General Data Protection Regulation](#) (GDPR) and therefore access cannot be granted at this time. For any issues, contact

# rest: **privire pragmatică**

## Raportarea erorilor

mesajele oferite trebuie să includă informații utile

exemplu:

GitHub – cod de stare HTTP oferit: 404

```
{  
  "message": "Not Found",  
  "documentation_url": "https://docs.github.com/en/rest"  
}
```

orice reprezentare oferită include minim un URL

rest: **privire pragmatică**

Controlul versiunilor API-ului dezvoltat

*“Never release an API without a version and make the version mandatory.” (Mulloy)*

considerații de interes în articolul J. Curry,  
*Introduction to API Versioning Best Practices* (2017):  
[nordicapis.com/introduction-to-api-versioning-best-practices/](https://nordicapis.com/introduction-to-api-versioning-best-practices/)

# rest: **privire pragmatică**

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

în antetul cererii HTTP

**Accept:** `application/vnd.heroku+json; version=3`

**Accept:** `application/vnd.discogs.v2.html+json`

# rest: **privire pragmatică**

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

în antetul cererii HTTP

în unele cazuri, folosind un antet propriu: **X-API-Version**

exemplu real: **X-GitHub-API-Version:2022-11-28**

versiuni incorecte conduc la întoarcerea codului HTTP **400**

# rest: **privire pragmatică**

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

în cadrul URL-ului – eventual, ca parametru

[api.cloudflare.com/client/v4/certificates/](https://api.cloudflare.com/client/v4/certificates/)

[wss://api.openai.com/v1/realtime?model=gpt-...](https://api.openai.com/v1/realtime?model=gpt-...)

[api.cognitive.microsofttranslator.com/translate?api-version=3.0](https://api.cognitive.microsofttranslator.com/translate?api-version=3.0)

[ec2.amazonaws.com/?Action=RunInstances&Version=2016-11-15](https://ec2.amazonaws.com/?Action=RunInstances&Version=2016-11-15)

# rest: **privire pragmatică**

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

## ***continuous versioning***

acces via același URI, indiferent de versiunea curentă

► practica preferată (“*Cool URIs don’t change*”)

[nordicapis.com/continuous-versioning-strategy-for-internal-apis/](http://nordicapis.com/continuous-versioning-strategy-for-internal-apis/)

# rest: **privire pragmatică**

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

## ***API Brownout***

deactivarea (unei versiuni a) unui API prin semnalarea (intermitentă) de erori pentru perioade scurte de timp

[apichangelog.substack.com/p/deactivating-an-api-one-step-at-a-time](https://apichangelog.substack.com/p/deactivating-an-api-one-step-at-a-time)  
[dev.to/martyndavies/what-does-api-brownout-actually-mean-2k4j](https://dev.to/martyndavies/what-does-api-brownout-actually-mean-2k4j)

caz real: Github

[github.blog/changelog/2021-08-10-brownout-notice-api-authentication-via-query-parameters-for-48-hours/](https://github.blog/changelog/2021-08-10-brownout-notice-api-authentication-via-query-parameters-for-48-hours/)



## rest: **privire pragmatică**

Paginarea și oferirea de răspunsuri parțiale

practică uzuală pentru paginare: **page** și **per\_page**

**/students/projects?page=2&per\_page=5**

alternativ, se pot folosi parametri precum **limit** și **offset**

**/students/projects?limit=33&offset=54**

**rest: privire pragmatică**

Paginarea și oferirea de răspunsuri parțiale

filtrele opționale pot fi delimitate de virgulă

**/students?fields=name,age,year,email**

## rest: **privire pragmatică**

Eterogenitatea formatelor reprezentărilor întoarse

indicarea formatului în URL via un parametru opțional

**?format=text/turtle** (DBpedia)

**?fmt=json** (MusicBrainz)

# rest: **privire pragmatică**

Eterogenitatea formatelor reprezentărilor întoarse

specificarea formatului acceptat în antetul cererii HTTP

**Accept: application/json** (BT Api Store)

**Accept: audio/wav** (IBM Watson Text to Speech)

# rest: **privire pragmatică**

Eterogenitatea formatelor reprezentărilor întoarse

precizarea formatului în numele resursei solicitate

**/venue.json** (Foursquare)

**/forecast.xml** (Weather)

# Movie Reviews API

DOWNLOAD SPECAUTHORIZE

Try this API

Request parameters

type

picks

offset

integer

✓ by-opening-date

by-publication-date

by-title

Credentials

apikey

EXECUTE

cURLHTTP

GET https://api.nytimes.com/svc/movies/v2/reviews/picks.json?order=by-opening-date

Accept: application/json

/movies/v2/reviews/picks.json?order=...

application/jsonRaw HTTP Response

{  
 "display\_title": "Parasite",  
 "mpaa\_rating": "R",  
 "critics\_pick": 1,  
 "byline": "Manohla Dargis",  
 "headline": "'Parasite' Review: The Lower Depths Rise With a Vengeance",  
 "summary\_short": "In Bong Joon Ho's new film, a destitute family occupies  
 "publication\_date": "2019-10-10",

răspunsul în  
format JSON

interogări interactive asupra serviciului de recenzie  
de filme (Movie Reviews) oferit de *The New York Times*  
[developer.nytimes.com](https://developer.nytimes.com)

# rest: **privire pragmatică**

Utilizarea subdomeniilor pentru API-uri diferite  
ale aceluiași ofertant de servicii

exemplificare:

**api.bing.microsoft.com**

**atlas.microsoft.com**

**graph.microsoft.com**

## rest: studiu de caz

Accesarea datelor publice despre Pokémon

colecții de resurse:

Pokémon, Locations, Contests, Games, Moves,...

fiecare categorie de resurse are proprietăți specifice  
*e.g.*, orice instanță de Locations include id, name, region etc.

detalii la [pokeapi.co](https://pokeapi.co)



Direct link to results: <https://pokeapi.co/api/v2/ability/battle-armor>

## Resource for battle-armor

răspuns disponibil  
în format JSON

```
effect_changes: [] 0 items
▼ effect_entries: [] 2 items
  ▼ 0: {} 3 keys
    effect: "Attacken können keinen kritischen Treffer landen. Diese"
    ▼ language: {} 2 keys
      name: "de"
      url: "https://pokeapi.co/api/v2/language/6/"
      short_effect: "Schützt vor kritischen Treffern."
    ▼ 1: {} 3 keys
      effect: "Moves cannot score critical hits against"
      ▼ language: {} 2 keys
        name: "en"
        url: "https://pokeapi.co/api/v2/language/9/"
        short_effect: "Protects against critical hits."
      ► flavor_text_entries: [] 65 items
    ▼ generation: {} 2 keys
      name: "generation-iii"
      url: "https://pokeapi.co/api/v2/generation/3/"
  id: 4
```

```
id: 6
iso3166: "de"
iso639: "de"
name: "de"
▼ names: [] 6 items
  ▼ 0: {} 2 keys
    ► language: {} 2 keys
      name: "ドイツ語"
    ► 1: {} 2 keys
  ▼ 2: {} 2 keys
    ▼ language: {} 2 keys
      name: "fr"
      url: "https://pokeapi.co/api/v2/language/5/"
      name: "Allemand"
  ▼ 3: {} 2 keys
    ▼ language: {} 2 keys
      name: "de"
      url: "https://pokeapi.co/api/v2/language/6/"
      name: "Deutsch"
```



Cum pot fi accesate  
reprezentări de resurse Web prin REST?

# rest: dezvoltare

## Navigatoarele Web actuale

nu necesită un API distinct pentru acces via HTTP

disponibilitate pe orice platformă

suport pentru REST prin obiectul **XMLHttpRequest**,  
și/sau API-uri HTML oferite de *browser*-ul Web:

**Fetch** ori **WebSocket**

[developer.mozilla.org/docs/Web/API](https://developer.mozilla.org/docs/Web/API)

# rest: dezvoltare

## Biblioteci/API-uri implementând HTTP

permit dezvoltarea de aplicații *desktop*, *mobile* etc.

suport pentru crearea de *mash-up*-uri Web  
la nivel de server

nu funcționează în navigatorul Web

**atenție la problemele de securitate ce pot apărea!**

# rest: dezvoltare – biblioteci (exemple)

**Akka HTTP** (Java, Scala): [doc.akka.io/docs/akka-http/](http://doc.akka.io/docs/akka-http/)

**Alamofire** (Swift): [github.com/Alamofire/Alamofire](https://github.com/Alamofire/Alamofire)

**http.client** (Python 3): [docs.python.org/3/library/http.client.html](https://docs.python.org/3/library/http.client.html)

**HTTP Toolkit** (soluție multi-platformă): [httptoolkit.tech](http://httptoolkit.tech)

**Httpful** (PHP): [phphttpclient.com](http://phphttpclient.com)

**khttp** (Kotlin + Java): [khttp.readthedocs.io](http://khttp.readthedocs.io)

**LibcURL** (C; portări Perl, PHP, Ruby,...): [curl.haxx.se/libcurl/](http://curl.haxx.se/libcurl/)

**node-fetch** (Node.js): [github.com/node-fetch/node-fetch](https://github.com/node-fetch/node-fetch)

**RestSharp** (.NET): [restsharp.dev](http://restsharp.dev)

**Requests** (Python): [requests.readthedocs.io](http://requests.readthedocs.io)

**Resty** (Go): [resty.dev](http://resty.dev)

# rest: dezvoltare – framework-uri (exemple)

ASP.NET MVC + Web API (C# *et al.*): [www.asp.net/web-api](http://www.asp.net/web-api)

Django, hug, restless (Python)

Express, LoopBack, Restify, Sails, Superagent (Node.js)

[www.npmjs.com/search?q=REST](http://www.npmjs.com/search?q=REST)

specificația JAX-RS (*Jakarta RESTful Web Services*): JSR-370

[projects.eclipse.org/projects/ee4j.rest](http://projects.eclipse.org/projects/ee4j.rest)

implementări: Apache CXF, Eclipse Jersey, EverRest, Jello

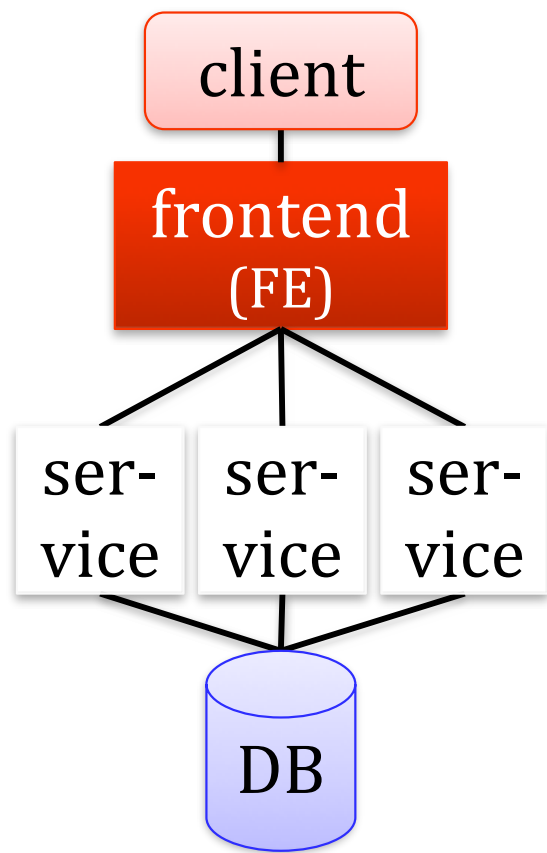
*Micro-framework-uri* PHP: Fat-Free, Lumen, Slim, Siler,...

de explorat și [github.com/marmelab/awesome-rest](https://github.com/marmelab/awesome-rest)

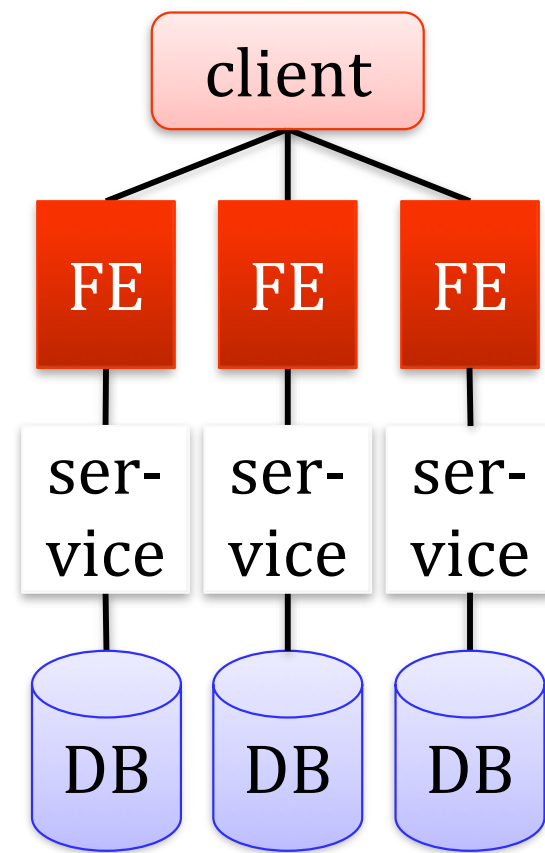
# rezumat

```
<I Hate Programming!>  
<I Hate Programming!>  
<I Hate Programming!>  
<IT WORKS!>  
<I LOVE Programming!>
```

aplicații orientate spre servicii Web  
de la HTTP la metodologii, instrumente și exemple



arhitectură bazată  
pe servicii Web



arhitectură recurgând  
la micro-servicii

episodul viitor:

**dezvoltarea de aplicații Web complexe**

specificarea API-urilor, micro-servicii,  
autentificare, autorizare, studii de caz